

PORTFOLIO

แฟ้มสะสมผลงานประกอบการสัมภาษณ์

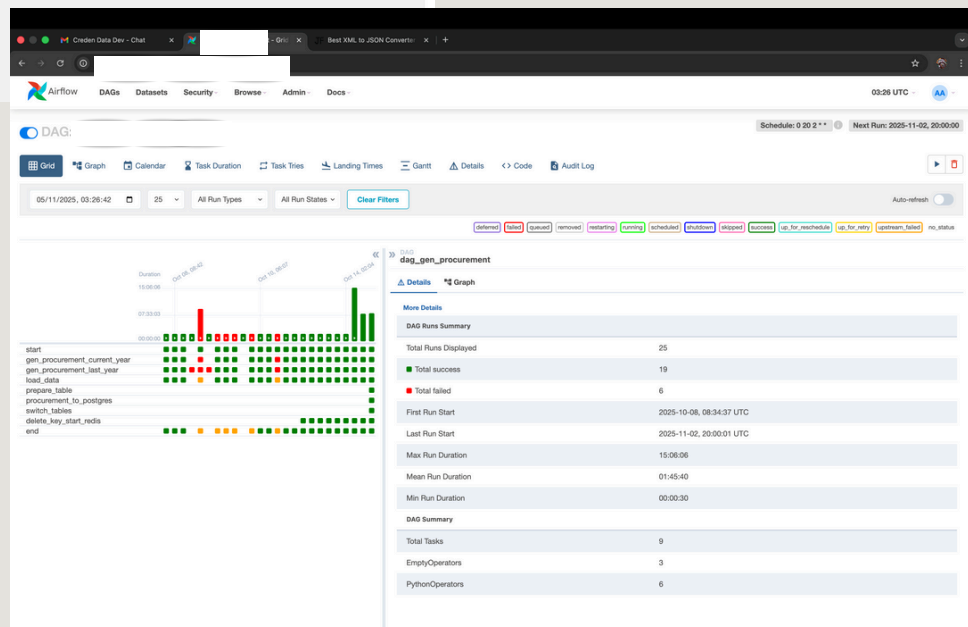
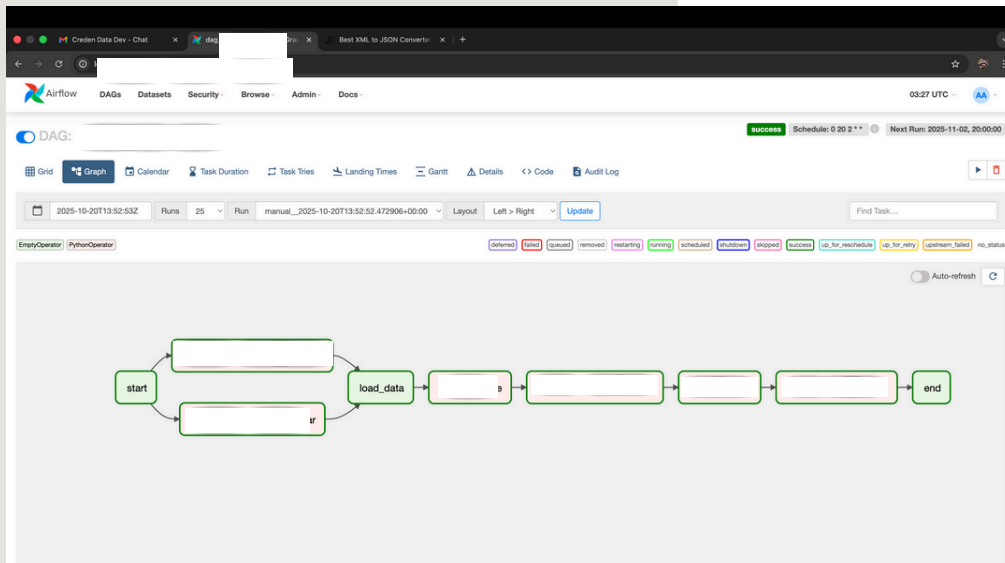
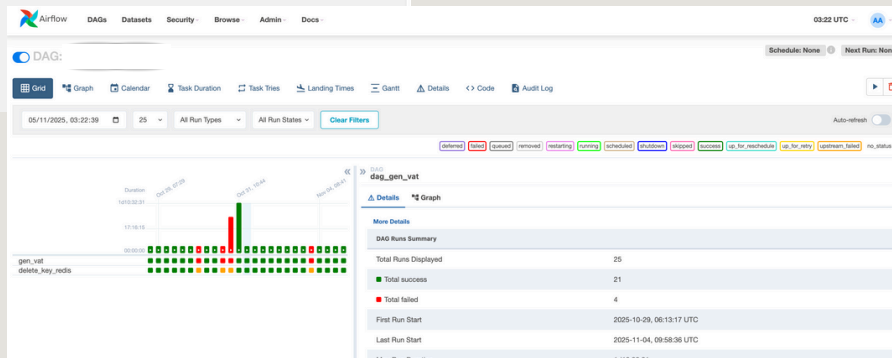
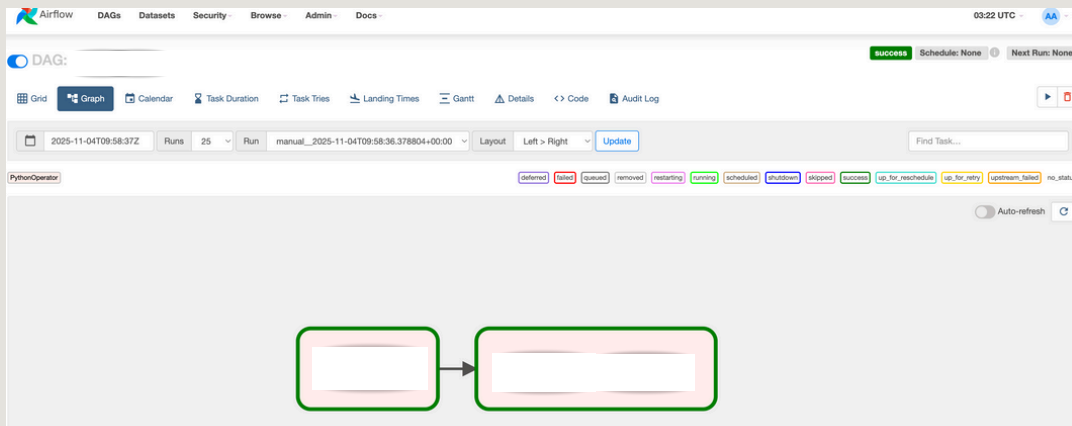


นายภูวดล วรบุตร

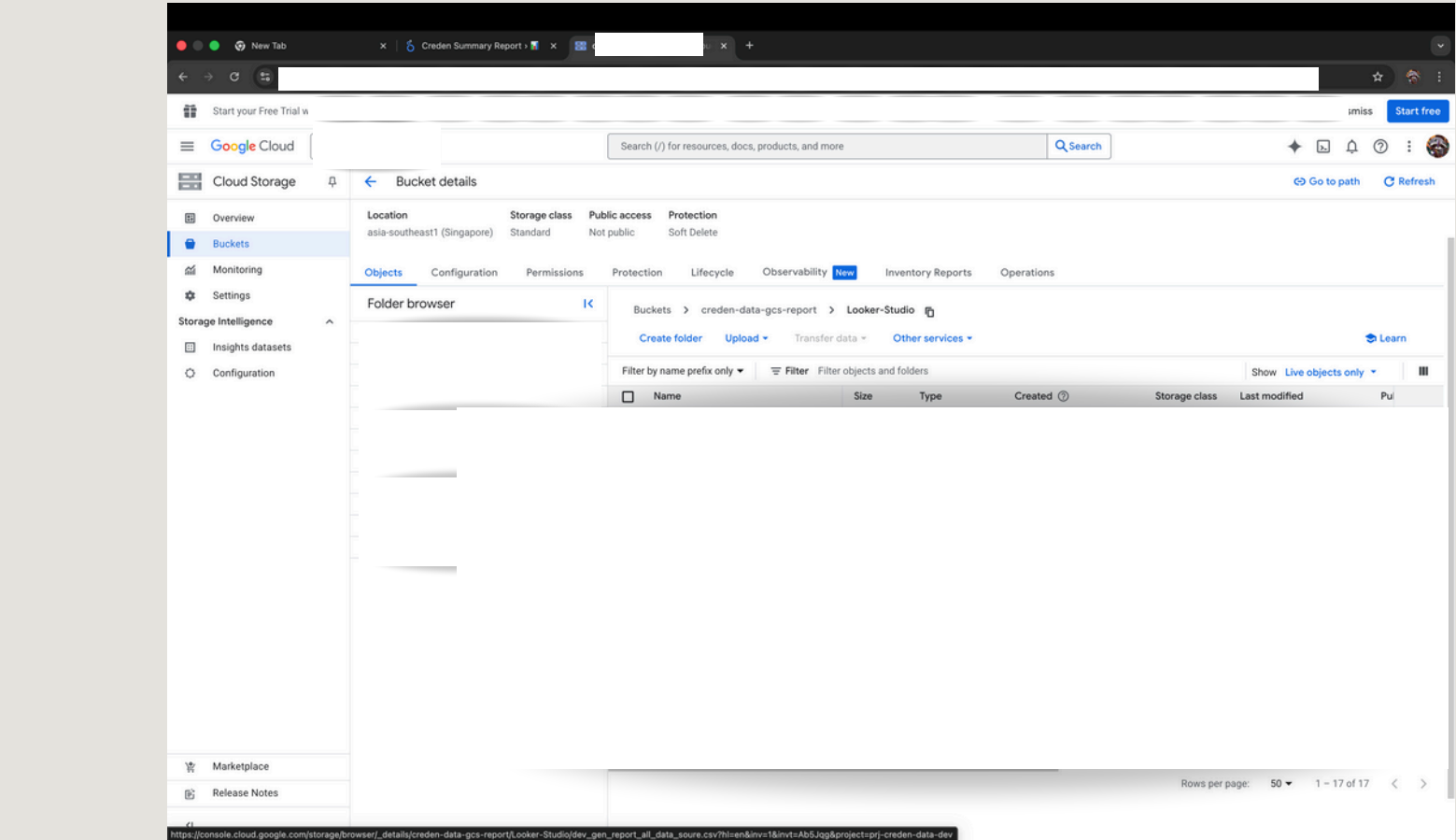
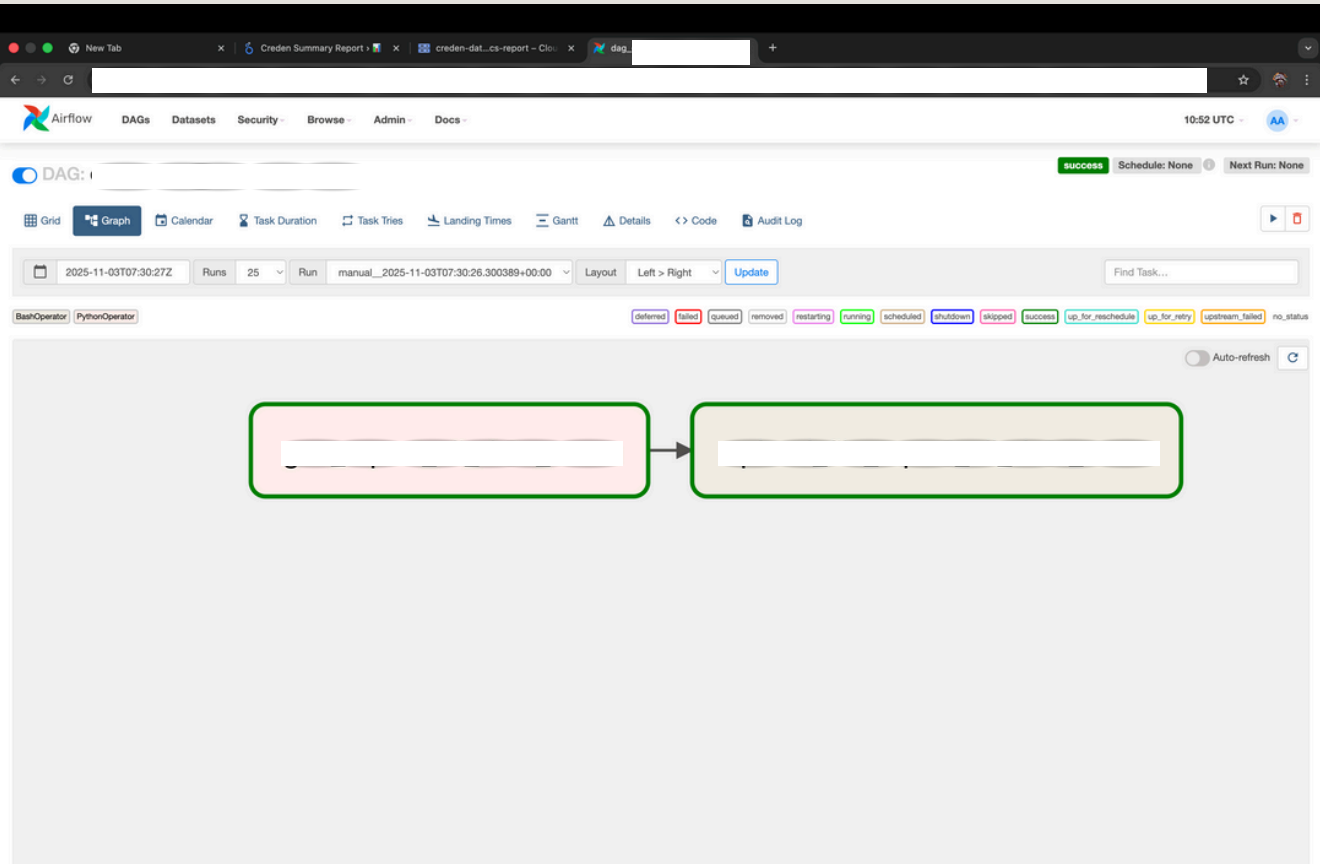
วิทยาศาสตร์บัณฑิต

มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ

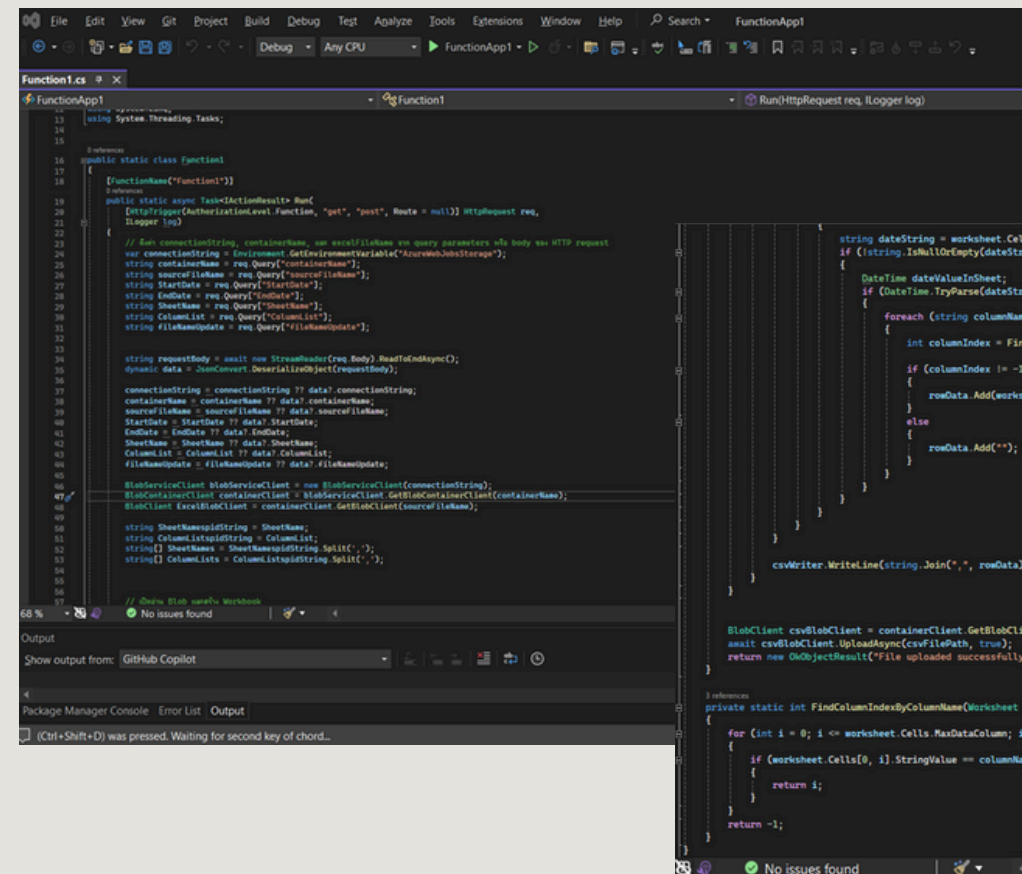
Apache Airflow



Automated Data Export from Database to GCS using Apache Airflow



Create C# Script on Azure Functions for transform data to csv format files



```
using System.Threading.Tasks;

namespace FunctionApp1
{
    [FunctionName("Function1")]
    public static async Task Run(
        [HttpTrigger(AuthorizationLevel.Function, "get", "post", Route = null)] HttpRequest req,
        ILogger log)
    {
        // Run connection string, containerName, and excelFileName as query parameters in body as HTTP request
        var connectionString = Environment.GetEnvironmentVariable("AzureWebJobsStorage");
        string containerName = req.Query["containerName"];
        string sourceFileName = req.Query["sourceFileName"];
        string startDate = req.Query["startDate"];
        string endDate = req.Query["endDate"];
        string sheetName = req.Query["sheetName"];
        string columnList = req.Query["columnList"];
        string fileNameUpdate = req.Query["fileNameUpdate"];

        string requestBody = await new StreamReader(req.Body).ReadToEndAsync();
        dynamic data = JsonConvert.DeserializeObject(requestBody);
        connectionString = data?.connectionString ?? data?.connectionString;
        containerName = data?.containerName ?? data?.containerName;
        sourceFileName = data?.sourceFileName ?? data?.sourceFileName;
        startDate = data?.startDate ?? data?.startDate;
        endDate = data?.endDate ?? data?.endDate;
        sheetName = data?.sheetName ?? data?.sheetName;
        columnList = data?.columnList ?? data?.columnList;
        fileNameUpdate = data?.fileNameUpdate ?? data?.fileNameUpdate;

        BlobServiceClient blobServiceClient = new BlobServiceClient(connectionString);
        BlobContainerClient containerClient = blobServiceClient.GetBlobContainerClient(containerName);
        BlobClient excelBlobClient = containerClient.GetBlobClient(sourceFileName);

        string sheetNameUpdate = sheetName;
        string columnListUpdate = columnList;
        string[] sheetNames = sheetNameUpdate.Split(';');
        string[] columnLists = columnListUpdate.Split(';');

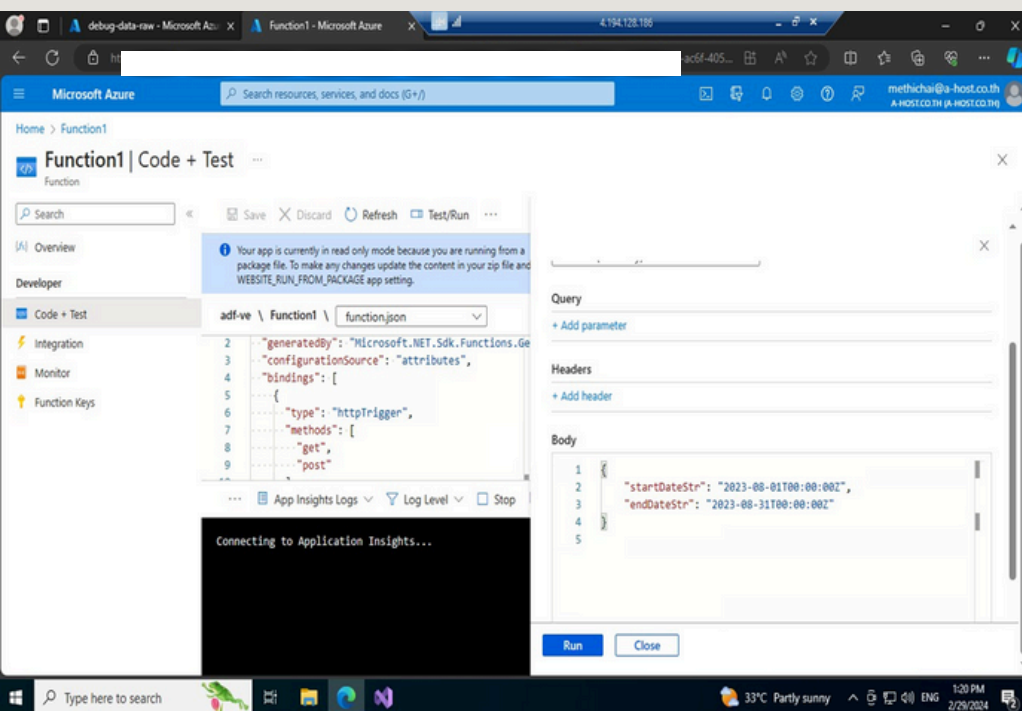
        // Open the excel file
        using var worksheet = excelBlobClient.DownloadToStream();
        using var csvWriter = new StreamWriter(containerClient.GetBlobClient(fileNameUpdate).OpenStream());

        string dataString = worksheet.Cells[row, FindColumnIndexByColumnName(worksheet, "Date")].Value?.ToString();
        if (!string.IsNullOrEmpty(dataString))
        {
            DateTime dateValueInSheet;
            if (DateTime.TryParse(dataString, out dateValueInSheet) && dateValue == dateValueInSheet)
            {
                foreach (string columnName in columns.Skip(1))
                {
                    int columnIndex = FindColumnIndexByColumnName(worksheet, columnName);
                    if (columnIndex != -1 && columnIndex <= worksheet.Cells.MaxDataColumn)
                    {
                        rowData.Add(worksheet.Cells[row, columnIndex].Value?.ToString() ?? "");
                    }
                    else
                    {
                        rowData.Add("");
                    }
                }

                csvWriter.WriteLine(string.Join(", ", rowData));
            }
        }

        BlobClient csvBlobClient = containerClient.GetBlobClient(csvFileName);
        await csvBlobClient.UploadAsync(csvFilePath, true);
        return new OkObjectResult("File uploaded successfully.");
    }

    private static int FindColumnIndexByColumnName(Worksheet worksheet, string columnName)
    {
        for (int i = 0; i <= worksheet.Cells.MaxDataColumn; i++)
        {
            if (worksheet.Cells[0, i].StringValue == columnName)
            {
                return i;
            }
        }
        return -1;
    }
}
```



Function1 | Code + Test

Search resources, services, and docs (G+)

Home > Function1

Function

Search

Save Discard Refresh Test/Run

Overview

Developer

Code + Test

Integration

Monitor

Function Keys

App Insights Logs Log Level Stop

Connecting to Application Insights...

Query

Add parameter

Headers

Add header

Body

```
1 {
2   "generatedBy": "Microsoft.NET.Sdk.Functions.Generator",
3   "configurationSource": "attributes",
4   "bindings": [
5     {
6       "type": "httpTrigger",
7       "direction": "in",
8       "methods": [
9         "get",
10        "post"
11      ]
12    }
13  ]
14 }
```

Run Close

Test/Run Upload

un.ps1

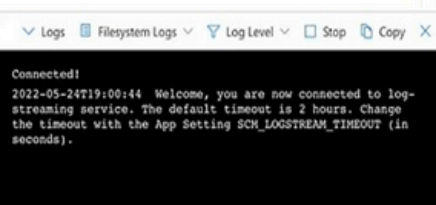
Input Output

HTTP response code

200 OK

HTTP response content

iis apppool\maws\placeholder105_f_v4_powershell_7_x86



Logs Filesystem Logs Log Level Stop Copy

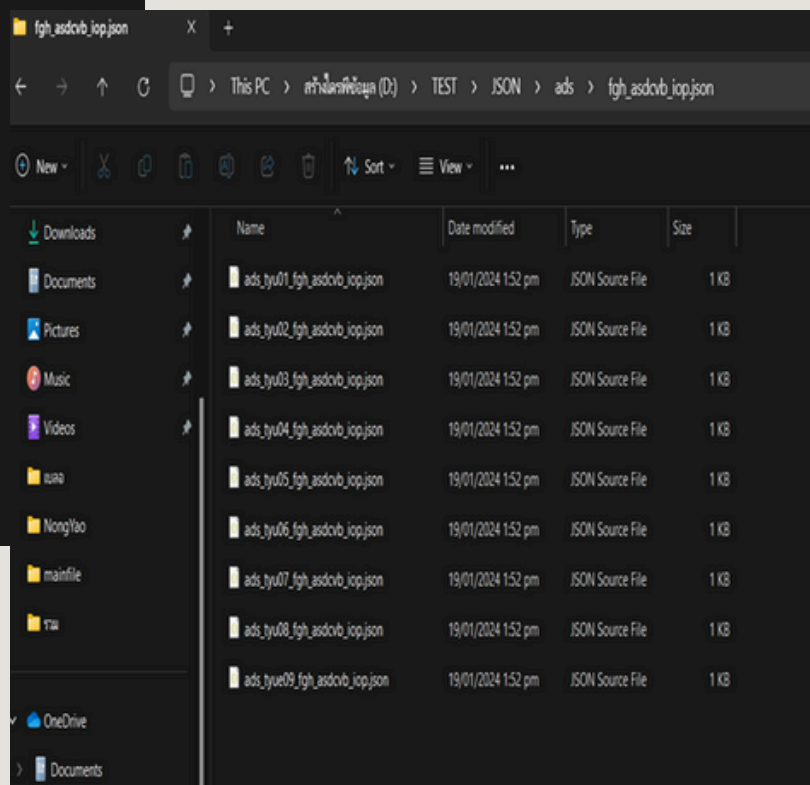
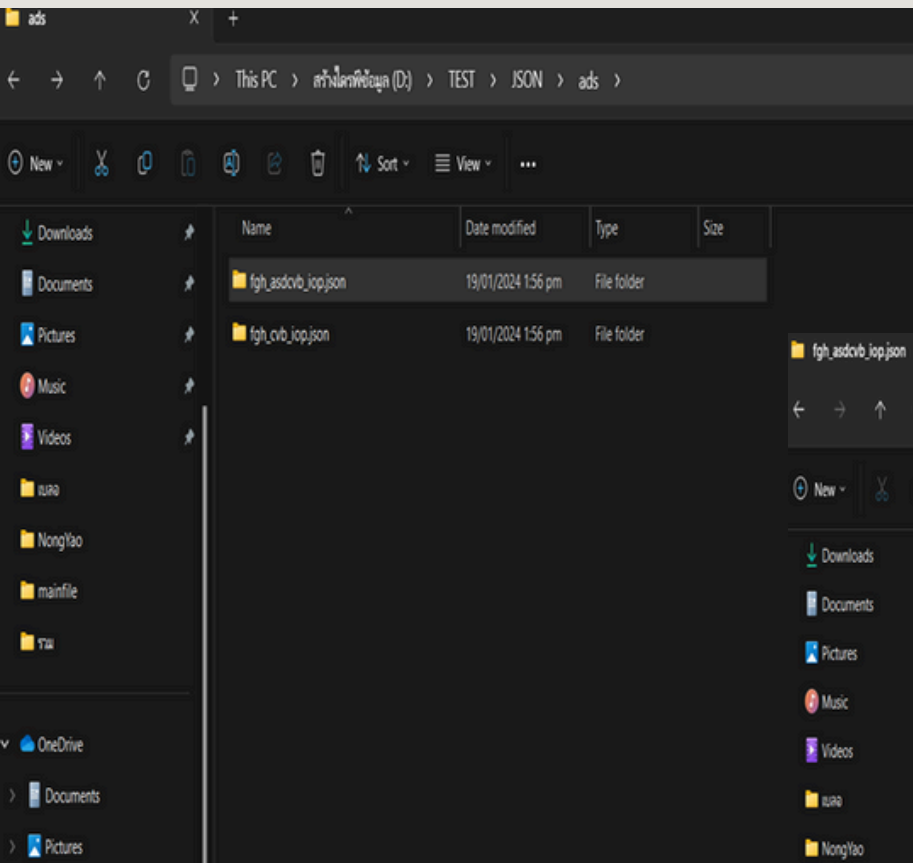
Connected!

2022-05-24T19:00:44 Welcome, you are now connected to log-streaming service. The default timeout is 2 hours. Change the timeout with the App Setting SCM_LOGSTREAM_TIMEOUT (in seconds).

Run Close

Create script for extract Json fromat files with Python

```
File Edit Selection View Go Run Terminal Help
Python_Batch > .py > get_destination_folder
1 import os
2 import shutil
3
4 def get_destination_folder(prefix, subfolder=None):
5     if subfolder:
6         return os.path.join(r'D:\TEST\JSON', prefix, subfolder)
7     else:
8         return os.path.join(r'D:\TEST\JSON', prefix)
9
10 def move_file(source_path, destination_path):
11     shutil.move(source_path, destination_path)
12
13 def move_files_to_folders(folder_path, prefixes):
14     for filename in os.listdir(folder_path):
15         file_path = os.path.join(folder_path, filename)
16         moved = False
17         for prefix in prefixes:
18             if filename.startswith(prefix):
19                 parts = filename.split('.')
20                 destination_folder = get_destination_folder(prefix, "-".join(parts[3:6]))
21                 destination_path = os.path.join(destination_folder, filename)
22                 if not os.path.exists(destination_folder):
23                     os.makedirs(destination_folder)
24                 move_file(file_path, destination_path)
25                 moved = True
26                 break
27         if not moved:
28             parts = filename.split('.')
29             destination_folder = get_destination_folder("others", "-".join(parts[3:]))
30             destination_path = os.path.join(destination_folder, filename)
31             if not os.path.exists(destination_folder):
32                 os.makedirs(destination_folder)
33             move_file(file_path, destination_path)
34
35 folder_path = r'D:\TEST\JSON\others'
36 # Prefixes ที่ต้องย้าย
37 prefixes_to_move = ['ads', 'ert', 'nty']
38
39 move_files_to_folders(folder_path, prefixes_to_move)
```



Association Rule Mining

```
%reset -f
import pandas as pd
import ast
import os
import numpy as np
from scipy import stats
from sqlalchemy import create_engine
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
from dotenv import load_dotenv
load_dotenv()
```

✓ 0.0s

True

```
def parse_braces_list(s):
    if pd.isna(s) or s == '{}':
        return []
    s = s.strip('{}')
    items = s.split(',')
    return [item.strip() for item in items]

def change_type(data_change_type):
    s = data_change_type.strip("[]").split(",")
    arr = [p.strip().strip('"').strip("'") for p in s]
    return arr

def get_menu_types(itemset, df_menu):
    types = []
    for code in itemset:
        match = df_menu.loc[df_menu["menu_code"] == code, "menu_type_code"]
        if not match.empty:
            types.append(match.values[0])
    return list(types)

def menu_type_dummy(all_menu_type, df_menu):
    menu_types = sorted(df_menu["menu_type_code"].unique())
    dummy = {t: 0 for t in menu_types}
    for t in all_menu_type:
        if t in dummy:
            dummy[t] += 1

    return dummy
```

```
def select_data_from_postgress(table_name):
    user = os.getenv("PG_USER")
    password = os.getenv("PG_PASSWORD")
    host = os.getenv("PG_HOST")
    port = os.getenv("PG_PORT")
    database = os.getenv("PG_DATABASE")
    connection_url = f"postgresql+psycopg2://{user}:{password}@{host}:{port}/{database}"
    engine = create_engine(connection_url)
    df = pd.read_sql_table(table_name, con=engine)
    if 'menu_name' in df.columns:
        df['menu_name'] = df['menu_name'].apply(parse_braces_list)
    if 'menu_type' in df.columns:
        df['menu_type'] = df['menu_type'].apply(parse_braces_list)
    return df
```

```
def filter_customer(count_customer):
    table_mapping = {
        1: ['customer_detail_a', 'customer_detail_b', 'customer_detail_c'],
        2: ['customer_detail_d', 'customer_detail_e', 'customer_detail_f', 'customer_detail_g', 'customer_detail_h'],
        3: ['customer_detail_i', 'customer_detail_j', 'customer_detail_k', 'customer_detail_l'],
        4: ['customer_detail_m', 'customer_detail_n'],
        5: ['customer_detail_o']
    }

    data_frames = []
    if count_customer not in table_mapping:
        print(f"ไม่พบบัญชีสำหรับ count_customer = {count_customer}")
        return pd.DataFrame()

    for table_name in table_mapping[count_customer]:
        try:
            df = select_data_from_postgress(table_name)
            if df is not None and not df.empty:
                data_frames.append(df)
        except Exception as e:
            print(f"❌ ผิดข้อมูลจาก {table_name} ไม่พบบัญชี: {e}")

    combined_df = pd.concat(data_frames, ignore_index=True)
    return combined_df
```

```
def outlier_price_interquartile_range(data):
    Q1 = data['price'].quantile(0.10)
    Q3 = data['price'].quantile(0.90)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    data_no_outlier = data[(data['price'] >= lower_bound) & (data['price'] <= upper_bound)].copy()
    return data_no_outlier
```

```
def transactionencoder(data_c):
    transactions = data_c['menu_name'].tolist()
    te = TransactionEncoder()
    te_ary = te.fit(transactions).transform(transactions)
    return pd.DataFrame(te_ary, columns=te.columns_)

def v_support(datatransaction , endcode_menu_name):
    frequent_itemsets = apriori(datatransaction, min_support=0.001, use_colnames=True)
    frequent_itemsets['menu_type'] = frequent_itemsets['itemsets'].apply(lambda x: get_menu_types(x, endcode_menu_name))
    frequent_itemsets['length_itemsets'] = frequent_itemsets['itemsets'].apply(lambda x: len(x))
    frequent_itemsets['length_menu_type'] = frequent_itemsets['menu_type'].apply(lambda x: len(x))
    return frequent_itemsets

def v_association_rules(value_support, endcode_menu_name):
    rules = association_rules(value_support, metric="confidence", min_threshold=0.001)
    rules["antecedents_length"] = rules["antecedents"].apply(lambda x: len(x))
    rules["consequents_length"] = rules["consequents"].apply(lambda x: len(x))
    rules = pd.concat([rules], axis=1)

    return rules.sort_values("support", ascending=False)
```

✓ 0.0s

```
endcode_menu_name = select_data_from_postgress('menu_name_table')

def process_association(table_name, filter_menu = None):
    if isinstance(table_name, int):
        g_data = filter_customer(table_name)
    else:
        g_data = select_data_from_postgress(table_name)
    g_data = g_data[g_data['menu_name'].apply(len) > 1]
    clear_outlier = outlier_price_interquartile_range(g_data)
    c_data = cleansing_data(clear_outlier)
    df_encoded = transactionencoder(c_data)
    frequent_itemsets = v_support(df_encoded , endcode_menu_name)
    rules = v_association_rules(frequent_itemsets, endcode_menu_name)

    if filter_menu :
        filtered = rules[
            rules['antecedents'].apply(lambda x: any(t in x for t in filter_menu)) |
            rules['consequents'].apply(lambda x: any(t in x for t in filter_menu))
        ]
    return filtered

return rules

data_association = process_association(3)
```

✓ 1m 42.6s

data_association

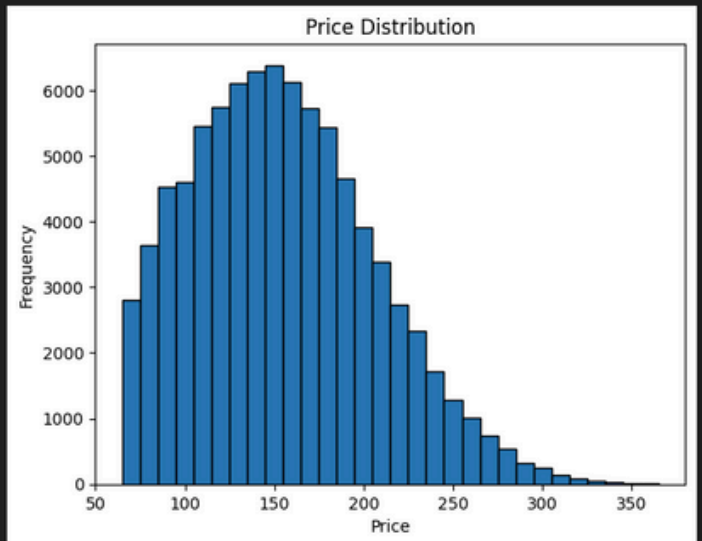
✓ 0.0s

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski	antecedents_length	consequents_length
539	(l)	(3)	0.115675	0.116223	0.012502	0.108080	0.929936	1.0	-0.000942	0.990870	-0.078509	0.056985	-0.009214	0.107825	1	
538	(3)	(l)	0.116223	0.115675	0.012502	0.107571	0.929936	1.0	-0.000942	0.990918	-0.078554	0.056985	-0.009165	0.107825	1	
1169	(f)	(u)	0.113328	0.118716	0.012288	0.108432	0.913367	1.0	-0.001166	0.988464	-0.096636	0.055918	-0.011670	0.105971	1	
1168	(u)	(f)	0.118716	0.113328	0.012288	0.103510	0.913367	1.0	-0.001166	0.989048	-0.097169	0.055918	-0.011073	0.105971	1	
907	(y)	(9)	0.113662	0.117960	0.012247	0.107746	0.913411	1.0	-0.001161	0.988553	-0.096620	0.055825	-0.011580	0.105783	1	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
22835	(a)	(g, c)	0.113365	0.011605	0.001001	0.008834	0.761189	1.0	-0.000314	0.997204	-0.261365	0.008078	-0.002804	0.047563	1	
22836	(c, i)	(a)	0.011657	0.113365	0.001001	0.085906	0.757783	1.0	-0.000320	0.969961	-0.244375	0.008075	-0.030970	0.047370	2	
22837	(c, a)	(i)	0.011730	0.112322	0.001001	0.085371	0.760060	1.0	-0.000316	0.970534	-0.242098	0.008138	-0.030361	0.047143	2	
22838	(i, a)	(c)	0.011454	0.115331	0.001001	0.087432	0.758092	1.0	-0.000320	0.969427	-0.244027	0.007962	-0.031537	0.048057	2	
15783	(7)	(4, e)	0.116823	0.011923	0.001001	0.008572	0.718947	1.0	-0.000391	0.996620	-0.306823	0.007839	-0.003392	0.046281	1	

30450 rows x 16 columns

```
import matplotlib.pyplot as plt
```

```
plt.hist(data_set_a['price'], bins=30, edgecolor='k')
plt.title('Price Distribution')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()
```



Project

การทำนายราคาทองคำด้วยการเรียนรู้ของเครื่องบนอนุกรมเวลาแบบแบน

วัตถุประสงค์

เพื่อออกแบบ พัฒนา และทดสอบแบบจำลองราคาทองคำรายวัน และวิเคราะห์ประเมินประสิทธิภาพของแบบจำลอง ตลอดจนเปรียบเทียบประสิทธิภาพของแบบจำลองการทำนาย ราคาทองคำแบบต่างๆ โดยใช้ข้อมูลย้อนหลัง 10 ปี จากเว็บทองคำ.com หรือ สมาคมค้าทองคำ และจะแบ่งข้อมูลเป็น 3 ส่วน เป็น 7 วัน 14 วัน และ 30 วันย้อนหลัง

อัลกอริทึมที่ใช้

- Decision Tree Regression
- Random Forest Regression
- Artificial Neural Network
- Linear Regression
- Lasso Regression

เครื่องมือในการดำเนินโครงการ

Microsoft Excel , Python , Google Colab
มีการใช้ไลบรารีจาก Scikit - Learn และ TensorFlow

ทดสอบโมเดลด้วย

MAE , MSE , RMSE และโมเดล Baseline และสร้างแบบจำลองการซื้อขายเพื่อดูกันว่าโมเดลที่ได้ค่า MAE , MSE , RMSE น้อยที่สุดหรือดีที่สุดจะได้กำไรหรือขาดทุน

```

df = pd.read_excel('/content/drive/MyDrive/Project/plt.xlsx')
x123 = df.iloc[:, 1]
x123_normalized = (x123 - x123.min()) / (x123.max() - x123.min())
USD = df.iloc[:, 2]
USD_normalized = (USD - USD.min()) / (USD.max() - USD.min())

plt.scatter(x123_normalized, USD_normalized, c=USD_normalized, label='USD')
plt.xlabel("Goldspot Data (Normalized)")
plt.ylabel("USD Data (Normalized)")
plt.title("Scatter Plot of Normalized Goldspot vs Normalized USD")
plt.colorbar(label='USD (Normalized)')
plt.legend()
plt.show()

GOLDSPOT = df.iloc[:, 1]
DATE = df.iloc[:, 0]
plt.figure(figsize=(20, 6))
plt.plot(DATE, GOLDSPOT)
plt.grid(True)
plt.xlabel("year")
plt.ylabel("gold spot")
plt.show()

# sns.pairplot(df)

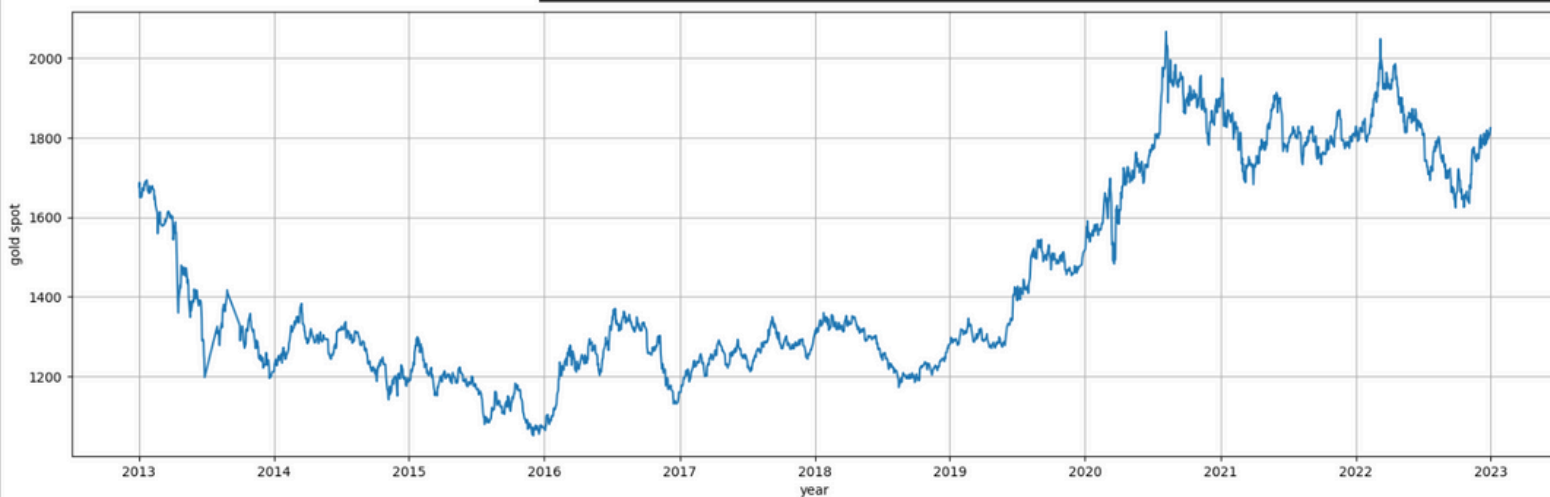
# train
data_train = pd.read_excel('/content/drive/MyDrive/Project/PDATA_Flannten_Time Series.xlsx', '7DAY')

# test
data_test = pd.read_excel('/content/drive/MyDrive/Project/PDATA_Flannten_Time Series.xlsx', 'DATA TEST 7DAY')

# train
train_target = data_train.iloc[:, 1] # Y
train_factor = data_train.iloc[:, 4:] # x

# test
test_target = data_test.iloc[:, 1] # Y
test_factor = data_test.iloc[:, 4:] # x

```



```

plt.figure(figsize=(15, 6))
plt.plot(test_target.values.flatten(), label='test_target')
plt.plot(predictions.flatten(), label='predictions')
plt.legend()
plt.show()

```

```

RES=test_target - predictions
sns.displot(RES,bins=30, kde=True)

```

```

lasso_model = Lasso(positive=True)
lasso_model.fit(train_factor, train_target)
predictions = lasso_model.predict(test_factor)
predictions = np.round(predictions, 2)

date = data_test.iloc[:, 0]

for idx, (d, target, pred) in enumerate(zip(date, test_target, predictions), start=1):
    date_str = d.strftime("%Y-%m-%d")
    # print(f"Row {idx}: Date:{date_str}, Actual: {target}, Predicted: {pred}")

mse = mean_squared_error(test_target, predictions)
print(f"MSE: {mse}")

mae = mean_absolute_error(test_target, predictions)
print(f"MAE: {mae}")

rmse = mean_squared_error(test_target, predictions, squared=False)
print(f"RMSE: {rmse}")

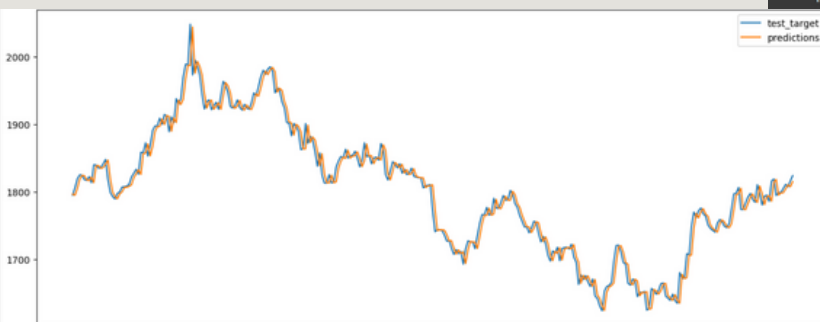
r_squared = r2_score(test_target, predictions)
print(f"R-squared: {r_squared}")

```

```

MSE: 223.2899517915311
MAE: 11.005667752443001
RMSE: 14.942889673404242

```



แบบจำลองการซื้อขาย

```
missed_opportunities_count = 0
no_trade_count = 0
buy_count = 0
profitable_buy_count = 0
loss_buy_count = 0
zero_profit_buy_count = 0
buy_sell_indicator_zero_count = 0

predictions = pd.DataFrame({'Predictions': predictions})
predictions = predictions.shift(periods=-1)
predictions = np.array(predictions)
predictions
Actual1 = test_target
Actual2 = test_target.shift(-1)

buy_sell = []
profits = []

for idx, (d, actual1, actual2, predictions) in enumerate(zip(date, Actual1, Actual2, predictions), start=1):
    date_str = d.strftime('%Y-%m-%d')

    if actual1 < predictions:
        buy_sell_indicator = 1
        profit = actual2 - actual1
        action_description = "ซื้อ"
    else:
        if actual1 < actual2:
            buy_sell_indicator = 0
            profit = 0
            action_description = "เสียโอกาส"
        else:
            buy_sell_indicator = 0
            profit = 0
            action_description = "ไม่ทำการซื้อขาย"

    buy_sell.append(buy_sell_indicator)
    profits.append(profit)

    if action_description == "เสียโอกาส":
        missed_opportunities_count += 1
    elif action_description == "ไม่ทำการซื้อขาย":
        no_trade_count += 1
    elif action_description == "ซื้อ":
        buy_count += 1
        if profit > 0:
            profitable_buy_count += 1
        elif profit < 0:
            loss_buy_count += 1
        if profit == 0:
            zero_profit_buy_count += 1
```

Row 290: Date: 2022-12-12,	Actual 1: 1789.0,	Actual 2: 1784.5,	Predictions tomorrow: [1789.16],	Buy/Sell: 1,	Profit: -4.5,	Action: ซื้อ
Row 291: Date: 2022-12-13,	Actual 1: 1784.5,	Actual 2: 1810.5,	Predictions tomorrow: [1784.31],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 292: Date: 2022-12-14,	Actual 1: 1810.5,	Actual 2: 1797.0,	Predictions tomorrow: [1808.72],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 293: Date: 2022-12-15,	Actual 1: 1797.0,	Actual 2: 1780.5,	Predictions tomorrow: [1797.19],	Buy/Sell: 1,	Profit: -16.5,	Action: ซื้อ
Row 294: Date: 2022-12-16,	Actual 1: 1780.5,	Actual 2: 1793.5,	Predictions tomorrow: [1781.],	Buy/Sell: 1,	Profit: 13.0,	Action: ซื้อ
Row 295: Date: 2022-12-17,	Actual 1: 1793.5,	Actual 2: 1795.0,	Predictions tomorrow: [1792.59],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 296: Date: 2022-12-19,	Actual 1: 1795.0,	Actual 2: 1786.0,	Predictions tomorrow: [1794.55],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 297: Date: 2022-12-20,	Actual 1: 1786.0,	Actual 2: 1816.0,	Predictions tomorrow: [1786.07],	Buy/Sell: 1,	Profit: 30.0,	Action: ซื้อ
Row 298: Date: 2022-12-21,	Actual 1: 1816.0,	Actual 2: 1819.0,	Predictions tomorrow: [1814.22],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 299: Date: 2022-12-22,	Actual 1: 1819.0,	Actual 2: 1794.5,	Predictions tomorrow: [1818.36],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 300: Date: 2022-12-23,	Actual 1: 1794.5,	Actual 2: 1799.0,	Predictions tomorrow: [1795.19],	Buy/Sell: 1,	Profit: 4.5,	Action: ซื้อ
Row 301: Date: 2022-12-24,	Actual 1: 1799.0,	Actual 2: 1799.0,	Predictions tomorrow: [1798.41],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 302: Date: 2022-12-26,	Actual 1: 1799.0,	Actual 2: 1805.0,	Predictions tomorrow: [1798.63],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 303: Date: 2022-12-27,	Actual 1: 1805.0,	Actual 2: 1811.0,	Predictions tomorrow: [1804.25],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 304: Date: 2022-12-28,	Actual 1: 1811.0,	Actual 2: 1807.5,	Predictions tomorrow: [1810.39],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 305: Date: 2022-12-29,	Actual 1: 1807.5,	Actual 2: 1816.0,	Predictions tomorrow: [1807.37],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 306: Date: 2022-12-30,	Actual 1: 1816.0,	Actual 2: 1824.0,	Predictions tomorrow: [1815.11],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 307: Date: 2022-12-31,	Actual 1: 1824.0,	Actual 2: nan,	Predictions tomorrow: [nan],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย

ผลรวมของกำไร: 73.0

จำนวนการซื้อ: 96

จำนวนการซื้อกำไร: 47

จำนวนการซื้อขาดทุน: 46

จำนวนการซื้อแต่ไม่ได้กำไร 3

จำนวนครั้งที่ไม่ซื้อ: 211

จำนวนครั้งที่ เสียโอกาส: 100

จำนวนครั้งที่ ไม่ทำการซื้อขาย: 111

Machine Learning Classification Model

ขั้นตอนที่ 1

เช็ค data

- เช็คค่า null
- describe() ตรวจสอบ outlier

```
data_credit = pd.read_csv('./Data Science Test/credit model test.csv')
print(f"isnull\n{data_credit.isnull().sum()}\n\n")
data_credit.describe()
```

```
isnull
Status of existing checking account      0
Duration in months                       0
Credit history                           0
Purpose                                  0
Credit amount                            0
Savings account/bonds                    0
Present employment since                  0
Installment rate in percentage of disposable income  0
Personal status and sex                   0
Other debtors / guarantors                0
Present residence since                    0
Property                                  0
Age in years                              0
Other installment plans                   0
Housing                                   0
Number of existing credits at this bank   0
Job                                         0
Number of people being liable to provide maintenance for  0
Telephone                                 0
Foreign worker                             0
Score                                      0
dtype: int64
```

	Duration in months	Credit amount	Installment rate in percentage of disposable income	Present residence since	Age in years	Number of existing credits at this bank	Number of people being liable to provide maintenance for
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	20.903000	3271.258000	2.973000	2.845000	35.546000	1.407000	1.155000
std	12.058814	2822.736876	1.118715	1.103718	11.375469	0.577654	0.362086
min	4.000000	250.000000	1.000000	1.000000	19.000000	1.000000	1.000000
25%	12.000000	1365.500000	2.000000	2.000000	27.000000	1.000000	1.000000
50%	18.000000	2319.500000	3.000000	3.000000	33.000000	1.000000	1.000000
75%	24.000000	3972.250000	4.000000	4.000000	42.000000	2.000000	1.000000
max	72.000000	18424.000000	4.000000	4.000000	75.000000	4.000000	2.000000

ขั้นตอนที่ 2

- ลบ outlier

```
def clear_outlier(data):
    numeric_cols = [
        'Duration in months',
        'Credit amount',
        'Installment rate in percentage of disposable income',
        'Present residence since',
        'Age in years',
        'Number of existing credits at this bank',
        'Number of people being liable to provide maintenance for'
    ]

    clean_data = data.copy()
    outlier_summary = {}

    for col in numeric_cols:
        Q1 = clean_data[col].quantile(0.25)
        Q3 = clean_data[col].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        # จำนวน outlier ก่อนลบ
        outliers = clean_data[(clean_data[col] < lower_bound) | (clean_data[col] > upper_bound)]
        outlier_summary[col] = len(outliers)

        # ลบ outliers แต่ละcolumn
        clean_data = clean_data[(clean_data[col] >= lower_bound) & (clean_data[col] <= upper_bound)].reset_index(drop=True)

    for col, count in outlier_summary.items():
        if count > 0:
            print(f"{col}: outliers = {count} rows")

    print("\ncount_all:", len(clean_data))
    return clean_data

data_clear_outlier = clear_outlier(data_credit)
data_clear_outlier.head()
```

Duration in months: outliers = 70 rows
Credit amount: outliers = 69 rows
Age in years: outliers = 31 rows
Number of existing credits at this bank: outliers = 4 rows
Number of people being liable to provide maintenance for: outliers = 124 rows

count_all: 702

	Status of existing checking account	Duration in months	Credit history	Purpose	Credit amount	Savings account/bonds	Present employment since	Installment rate in percentage of disposable income	Personal status and sex	Other debtors / guarantors	...	Property
0	No checking account	24	existing credits paid back duly till now	furniture/equipment	2835	500 to 1000	more than 7 years	3	male (single)	none	...	building society savings agreement/ life insur...
1	0 - 200	36	existing credits paid back duly till now	car (used)	6948	less than 100	1 to 4 years	2	male (single)	none	...	car or other
2	No checking account	12	existing credits paid back duly till now	radio/television	3059	more than 1000	4 to 7 years	2	male (divorced/separated)	none	...	real estate
3	0 - 200	30	critical account/ other credits existing (not ...	car (new)	5234	less than 100	Unemployed	4	male (married/widowed)	none	...	car or other
4	0 - 200	12	existing credits paid back duly till now	car (new)	1295	less than 100	less than 1 year	3	female (divorced/separated/married)	none	...	car or other

5 rows x 21 columns

ขั้นตอนที่ 3

เช็ค Correlation

- เพื่อ วัดความสัมพันธ์ระหว่างตัวแปร กับ target ถ้าใกล้ 0 ตัดออก เพื่อลด Noise ทำให้โมเดล นิ่งขึ้น ไม่ลวน ไม่ overfit
- และ เลือก column ที่ส่งผล กับ target

```
# Correlation วัดความสัมพันธ์ระหว่างตัวแปร กับ Score ถ้าใกล้ 0 ตัดออก เพื่อลด Noise ทำให้โมเดล นิ่งขึ้น ไม่ลวน ไม่ overfit
def classify_feature(corr_value):
    abs_corr = abs(corr_value)
    if abs_corr > 0.1:
        return "get"
    else:
        return "remove"

# ทำ copy ของ dataset
df_copy = data_credit.copy()

encoders = {}
cat_cols = df_copy.select_dtypes(include=['object']).columns
for col in cat_cols:
    le = LabelEncoder()
    df_copy[col] = le.fit_transform(df_copy[col])
    encoders[col] = le

# คำนวณ correlation matrix
corr_matrix = df_copy.corr()

# ดู correlation กับ Score
score_corr = corr_matrix['Score'].sort_values(ascending=False)

corr_df = pd.DataFrame({
    'Feature': score_corr.index,
    'Correlation_with_Score': score_corr.values,
    'Action': [classify_feature(x) for x in score_corr.values]
})

selected_features = corr_df['Feature'][corr_df['Action'] == 'get'].tolist()
data_final = data_clear_outlier[selected_features].copy()

for col in cat_cols:
    if col in data_final.columns:
        data_final[col] = encoders[col].transform(data_final[col])

corr_df
```

	Feature	Correlation_with_Score	Action
0	Score	1.000000	get
1	Status of existing checking account	0.251307	get
2	Savings account/bonds	0.103133	get
3	Age in years	0.091127	remove
4	Personal status and sex	0.089535	remove
5	Purpose	0.089096	remove
6	Other installment plans	0.053862	remove
7	Number of existing credits at this bank	0.045732	remove
8	Telephone	0.036466	remove
9	Job	0.032756	remove
10	Other debtors / guarantors	0.028441	remove
11	Number of people being liable to provide maint...	0.003015	remove
12	Present residence since	-0.002967	remove
13	Present employment since	-0.004263	remove
14	Housing	-0.019315	remove
15	Property	-0.034011	remove
16	Installment rate in percentage of disposable i...	-0.072404	remove
17	Foreign worker	-0.082079	remove
18	Credit history	-0.097054	remove
19	Credit amount	-0.154739	get
20	Duration in months	-0.214927	get

ขั้นตอนที่ 4

- แบ่งตัวแปรต้นและตัวแปรตาม
- และแบ่ง ตัวที่เอาไว้ train และ test โดยแบ่งเป็น 80:20 เพราะใช้จากจำนวน data มีประมาณนี้

ขั้นตอนที่ 5

- เตรียม Model RandomForest เนื่องจาก ตัวแปรตามเป็นแค่ Binary Classification เลยใช้ RandomForest เพราะว่ามันเป็น decision tree หลายต้น

ขั้นตอนที่ 6

- train Model
- และ predict

```
X = data_final.drop('Score', axis=1)
y = data_final['Score']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

rf = RandomForestClassifier(
    n_estimators=500,
    max_depth=20,
    min_samples_split=5,
    min_samples_leaf=2,
    random_state=42
)

rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)

results = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred}).reset_index(drop=True)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred, labels=rf.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=rf.classes_)
disp.plot(cmap=plt.cm.Blues)
plt.title("Confusion Matrix")
plt.show()

results.head(20)
```

	Actual	Predicted
0	1	1
1	1	1
2	0	1
3	1	1
4	1	1
5	1	1
6	1	1
7	1	1
8	1	1
9	1	1
10	1	1
11	1	1
12	1	1
13	1	1
14	1	1

ขั้นตอนที่ 7

- วัดความแม่นยำ accuracy โดยค่าที่ได้คือ 76%
- และแสดง กราฟ

โดยที่กราฟ บ่งบอกว่า

Class 0

ถูกต้อง 11 แถว

พลาด 26 แถว (โมเดลไม่แม่นยำสำหรับ class 0)

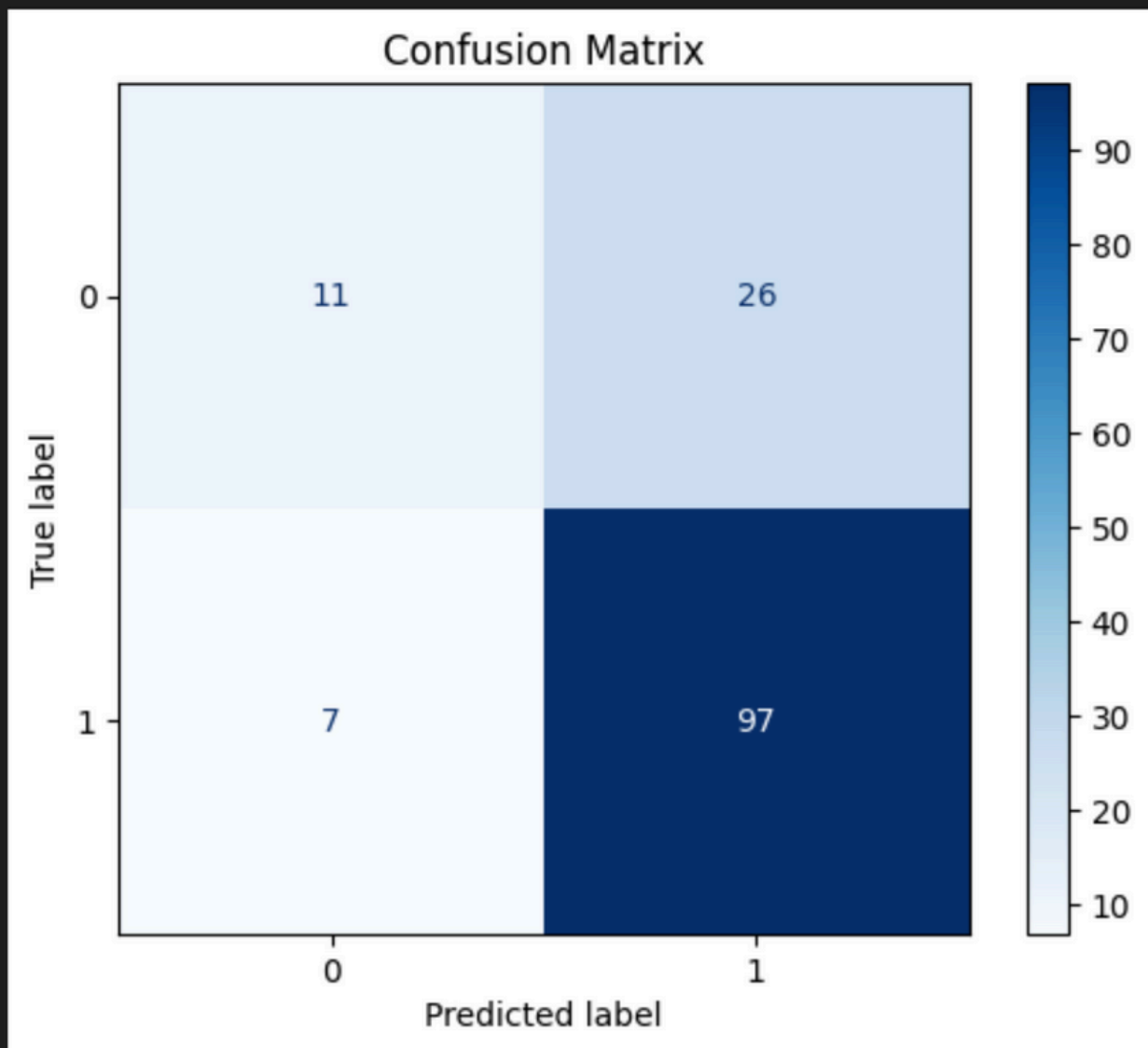
Class 1

ถูกต้อง 97 แถว

พลาด 7 แถว

โมเดลแม่นยำสำหรับ class 1

Accuracy: 0.7659574468085106



Machine Learning Clustering Model

ขั้นตอนที่ 1

- ดึง data
- เช็ค data
- เช็คค่า null
- เช็ค describe เพื่อดู scale

```
%reset -f
import pandas as pd
import numpy as np
from sklearn.preprocessing import RobustScaler
from sklearn.cluster import DBSCAN
```

```
# จัดกลุ่มตามความเสี่ยงทางการเงิน
data_opportunity = pd.read_csv('./Data Science Test/Opportunity.csv')
features = ['cifno', '2023+D/E_ratio', '2023+liquidity_ratio', '2023+current_assets', '2023+current_liability', '2023+total_assets', '2023+total_liabilities']
data_opportunity = data_opportunity[features]
print(f"isNull\n{data_opportunity.isnull().sum()}\n\n")
data_opportunity.describe()
```

```
isNull
cifno                0
2023+D/E_ratio       0
2023+liquidity_ratio  0
2023+current_assets  0
2023+current_liability 0
2023+total_assets    0
2023+total_liabilities 0
dtype: int64
```

	2023+D/E_ratio	2023+liquidity_ratio	2023+current_assets	2023+current_liability	2023+total_assets	2023+total_liabilities
count	59877.000000	59877.000000	5.987700e+04	5.987700e+04	5.987700e+04	5.987700e+04
mean	0.347249	1.319964	3.530631e+06	1.549367e+06	6.395217e+06	3.157399e+06
std	0.863030	3.102715	7.306187e+06	3.870246e+06	1.263088e+07	7.425729e+06
min	-2.690000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
25%	0.000000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
50%	0.000000	0.000000	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
75%	0.220000	1.190000	3.441820e+06	6.213613e+05	7.092182e+06	1.831609e+06
max	4.750000	21.450000	4.121130e+07	2.377186e+07	7.023691e+07	4.380579e+07

ขั้นตอนที่ 2

- cleansing
- เช็ค column ที่มี < 0
- ปรับ column ที่มี $< 0 == 0$
- ปรับ ค่าความสมมาตร ลดความ skew (ความเอียงของข้อมูล) financial ที่กระจายกว้าง ทำให้มัน สมมาตร เพื่อ clustering แม่นขึ้น
- ปรับ ข้อมูลทุก column อยู่ scale ใกล้เคียงกัน เปลี่ยนค่าใหม่ทุก column

```
df = data_opportunity.copy()
ids = df['cifno']

features_not_id = ['2023+D/E_ratio', '2023+liquidity_ratio', '2023+current_assets', '2023+current_liability', '2023+total_assets', '2023+total_liabilities']

print("Negative values per column:\n", (df[features_not_id] < 0).sum())

df["2023+D/E_ratio"] = df["2023+D/E_ratio"].clip(lower=0)
df_log = df[features_not_id].apply(np.log1p) # ลดความ skew (ความเอียงของข้อมูล) financial ที่กระจายกว้าง ทำให้มัน สมมาตร

# RobustScaler
scaler = RobustScaler() # ทำให้ข้อมูลทุก column อยู่ scale ใกล้เคียงกัน เปลี่ยนค่าใหม่ทุก column
df_scaled = pd.DataFrame(
    scaler.fit_transform(df_log),
    columns=features_not_id
)

df_scaled['cifno'] = ids
df_scaled
```

Negative values per column:

```
2023+D/E_ratio      736
2023+liquidity_ratio      0
2023+current_assets      0
2023+current_liability    0
2023+total_assets        0
2023+total_liabilities    0
dtype: int64
```

	2023+D/E_ratio	2023+liquidity_ratio	2023+current_assets	2023+current_liability	2023+total_assets	2023+total_liabilities	
0	0.000000	0.363794	1.152134	0.000000	0.000000	0.000000	3051bc859a48c9de8a2d5746d5
1	0.000000	1.290469	1.050514	1.143483	1.085576	1.180277	86e498e59c6f7f742901a76db0c
2	8.752621	0.567273	0.000000	0.000000	0.000000	0.000000	3d13ab525597c5eb6310872345f
3	0.000000	3.312592	1.056629	1.003387	1.014380	1.108342	38307e63274d01eb0186b0d2db
4	8.083626	0.756889	0.000000	0.000000	0.000000	0.000000	744db712e7c95df067d0f9726a5
...	...	...	...	...	...	...	...
59872	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	a1c3638dd918aba11b95369c8b8:
59873	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	ca9b3ad5c90f209a0be18e35a13
59874	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	92619667ee1de235efe8299c3e:
59875	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	6adaf802e57977d2d1ae589f6b:
59876	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	57bd225ca00255b82d686553709

59877 rows x 7 columns

ขั้นตอนที่ 3

- ใช้ model DBSCAN เพราะอยากให้มันกำหนดกลุ่มเองด้วยความถี่
- ปรับ eps=1 เพื่อให้ระยะห่างระหว่างกลุ่ม ไม่น้อยไปกว่า 1
- ปรับ min_samples=10 ให้ data อย่างน้อยจับคู่กัน 10 กลุ่มขึ้นไป
- เลือก metric วัดระยะห่างระหว่างจุดข้อมูล ด้วย เส้นตรง
- และ predict

```
cluster_features = ['2023+D/E_ratio', '2023+liquidity_ratio', '2023+current_assets', '2023+current_liability', '2023+total_assets', '2023+total_liabilities']

dbscan = DBSCAN(eps=1, min_samples=10, metric='euclidean')

df_scaled['cluster'] = dbscan.fit_predict(df_scaled[cluster_features])

df_scaled['cluster'].value_counts()
```

```
cluster
0    59273
3     143
7      99
2      71
5      71
1      55
8      48
4      44
6      42
-1     31
```

ขั้นตอนที่ 4

- เช็ค่าความเสี่ยงของแต่ละกลุ่ม

โดยที่

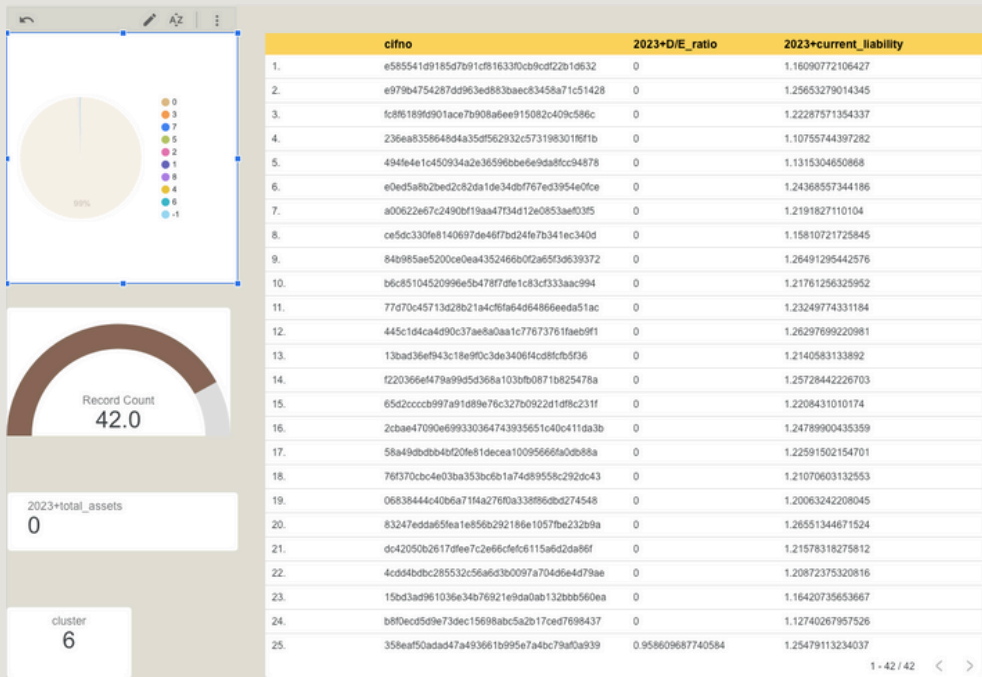
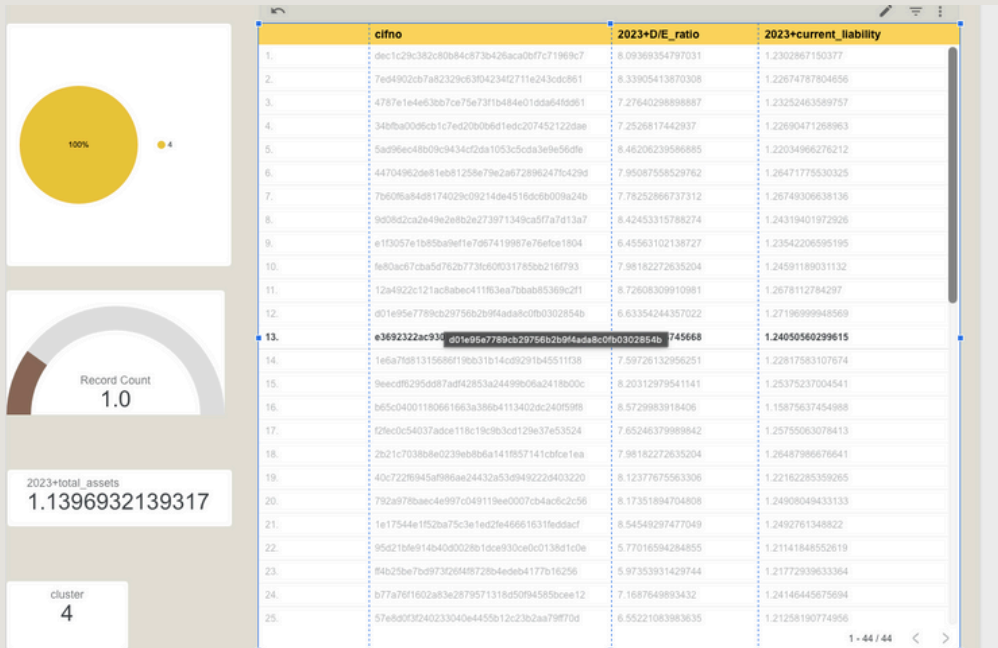
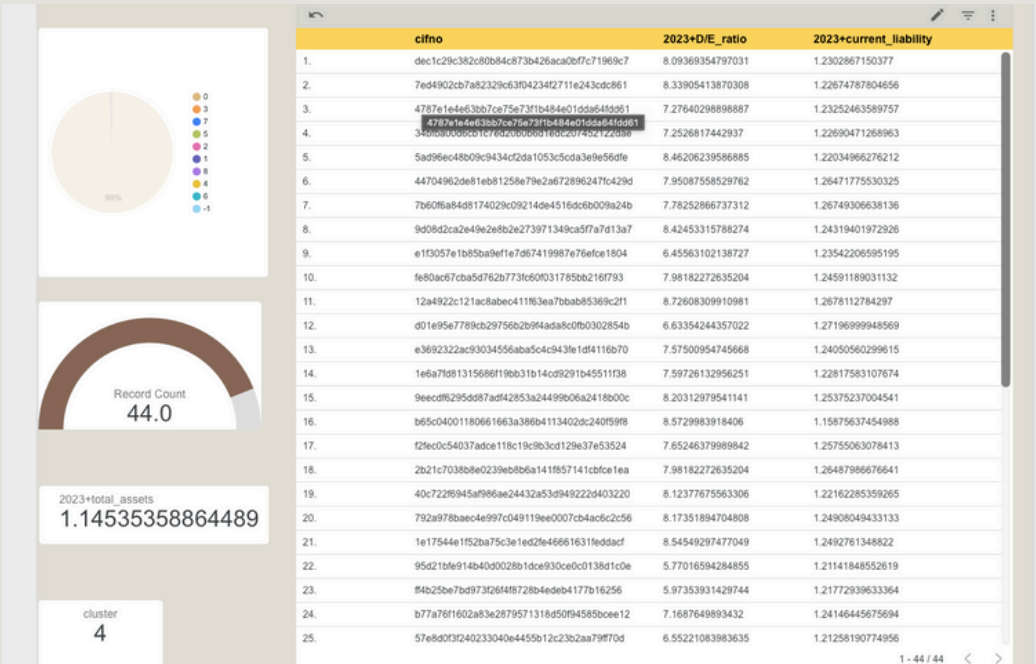
Cluster	ความเสี่ยง
4	**สูงสุด** D/E สูง, liquidity ต่ำ
2	สูง D/E สูง, liquidity ปานกลาง
1	สูง D/E สูง, liquidity ต่ำ
3	ปานกลาง D/E สูง, total liabilities สูง
5	ปานกลาง/ต่ำ liquidity สูง
7	ต่ำ D/E ต่ำ
8	ปานกลาง liquidity ต่ำ
0	ต่ำ ข้อมูล null เยอะ
6	ต่ำ D/E 0, liquidity สูง

```
cluster_summary = df_scaled.groupby('cluster')[cluster_features].median().sort_values('2023+D/E_ratio', ascending=False)
cluster_summary
```

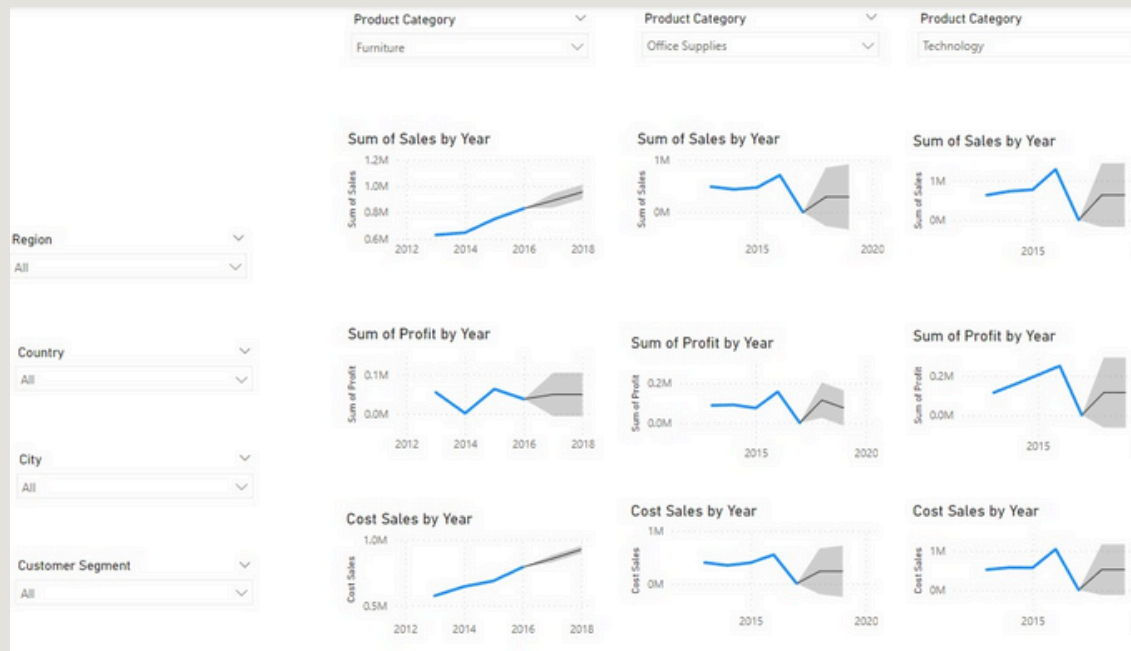
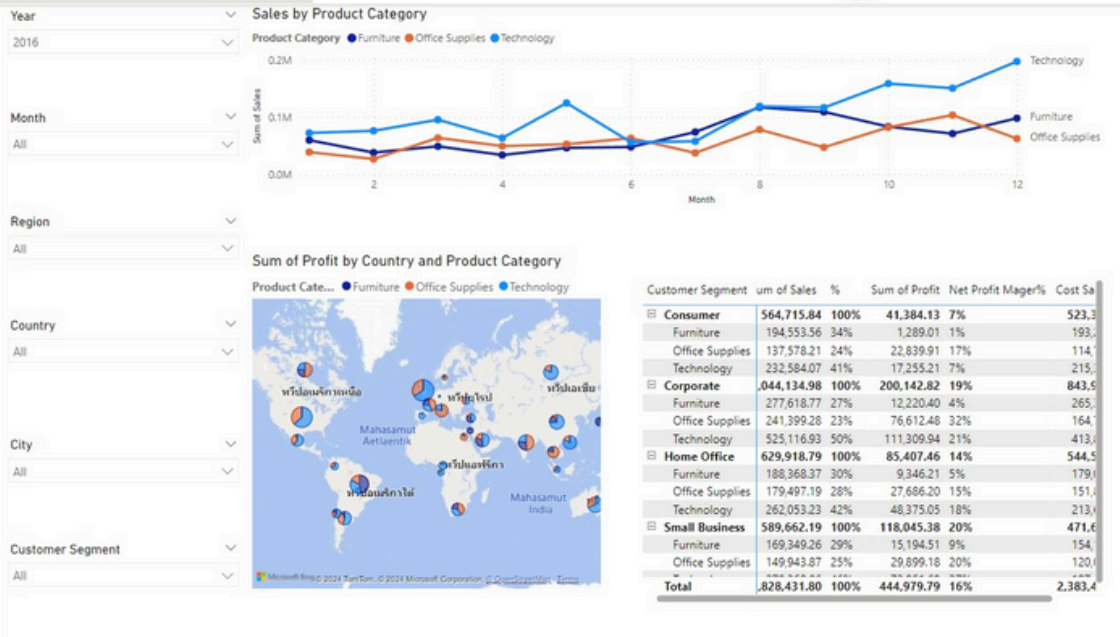
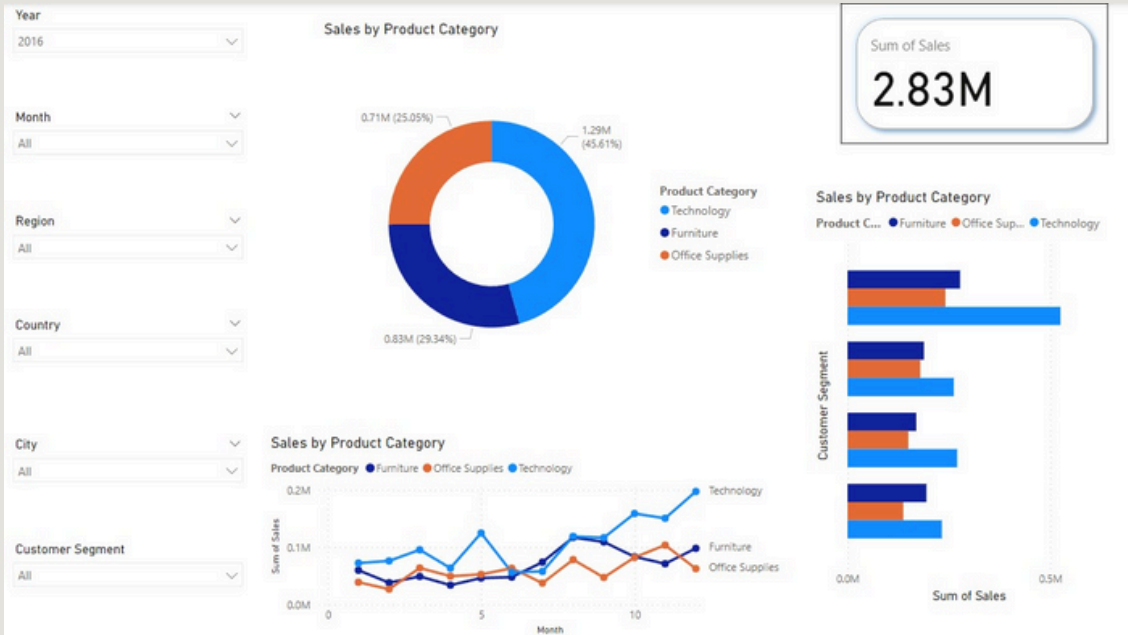
	2023+D/E_ratio	2023+liquidity_ratio	2023+current_assets	2023+current_liability	2023+total_assets	2023+total_liabilities
cluster						
4	7.575010	0.927978	1.107904	1.246480	1.137683	0.000000
2	7.462249	1.106134	0.000000	0.000000	1.141553	0.000000
1	7.083424	0.713885	1.139207	0.000000	1.140419	0.000000
-1	6.019326	1.626943	0.000000	1.169590	1.113276	0.000000
3	5.423208	1.238394	0.000000	0.000000	1.130329	1.208984
5	4.957569	2.085856	0.000000	1.253940	0.000000	0.000000
7	2.638804	1.546037	0.000000	0.000000	0.000000	1.211629
8	2.394834	0.676818	1.113179	0.000000	0.000000	1.205499
0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
6	0.000000	2.564405	0.000000	1.217548	0.000000	0.000000

ขั้นตอนที่ 5

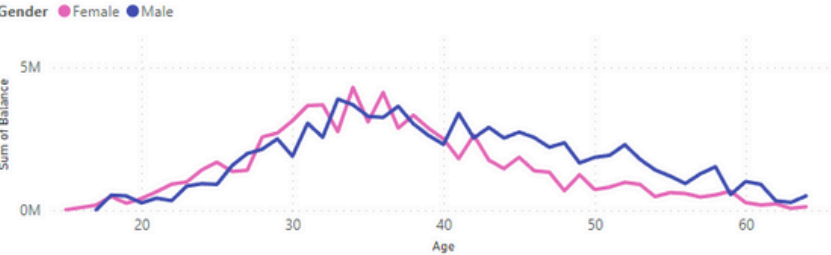
- แสดง กราฟจาก looker studio



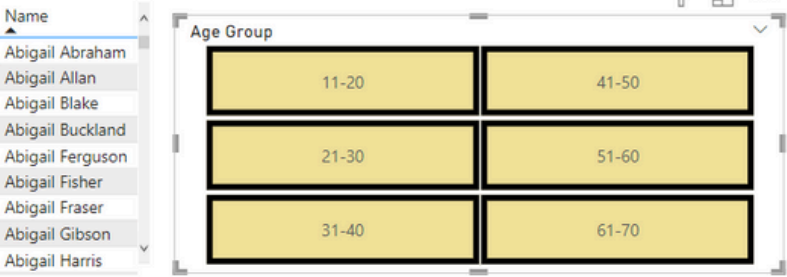
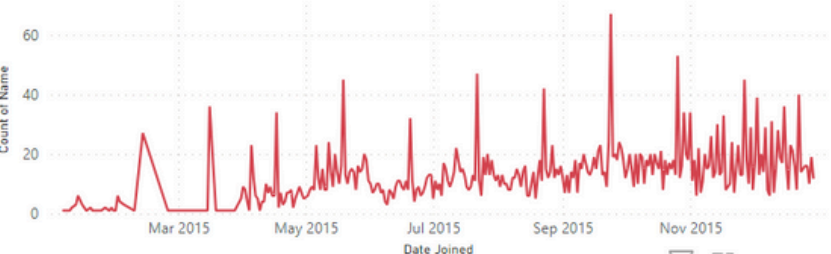
Data Visualization



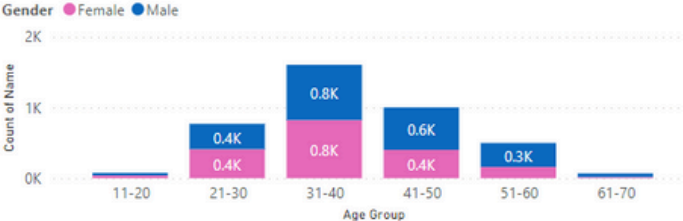
Balance by Age and Gender



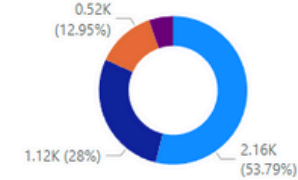
Total Customer



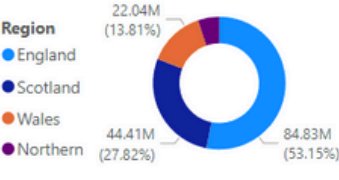
Number of Customer by Age and Gender



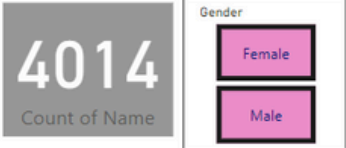
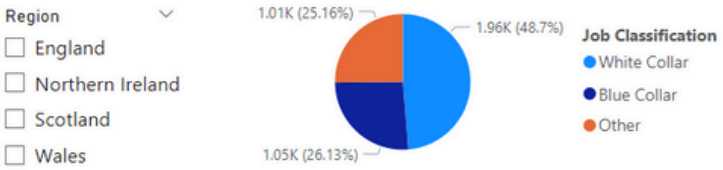
Count of Name by Region



Balance by Region



Number Customer by Job

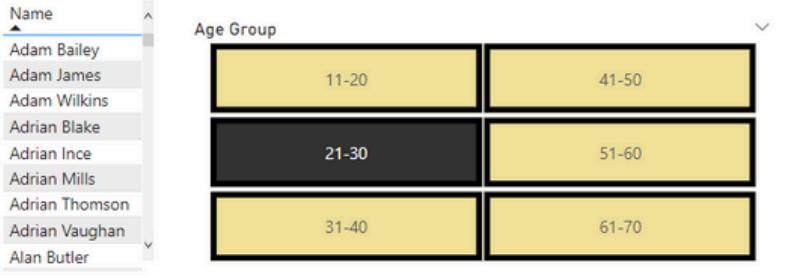
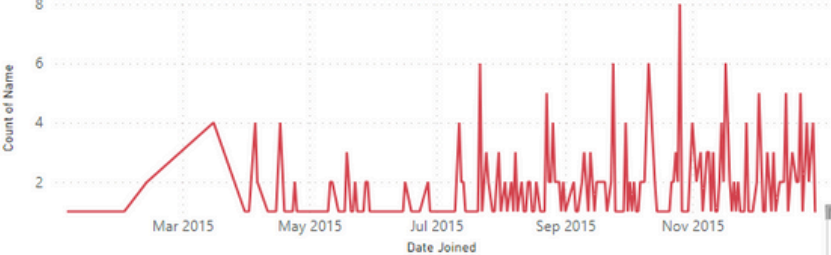


Bank Data Analytics

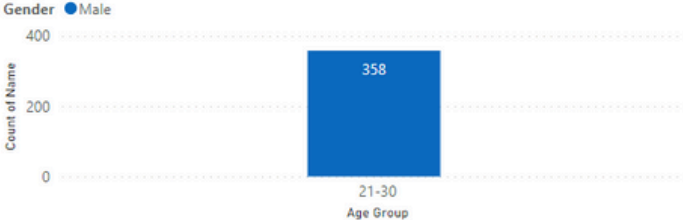
Balance by Age and Gender



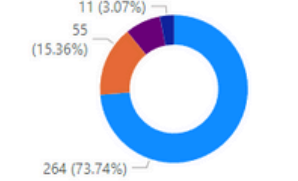
Total Customer



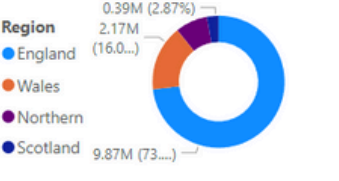
Number of Customer by Age and Gender



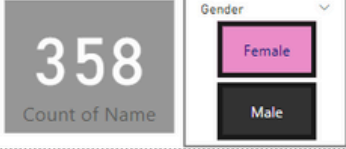
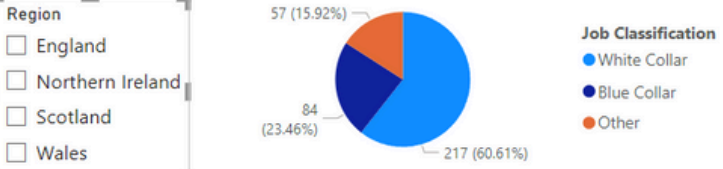
Count of Name by Region



Balance by Region



Number Customer by Job



Bank Data Analytics

Python Data Visualization

```
%reset -f
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ 0.9s

```
Orders = pd.read_excel('./Data Science Test/Sample - Superstore.xls', engine="xlrd", sheet_name='Orders')
Returns = pd.read_excel('./Data Science Test/Sample - Superstore.xls', engine="xlrd", sheet_name='Returns')
People = pd.read_excel('./Data Science Test/Sample - Superstore.xls', engine="xlrd", sheet_name='People')
```

✓ 0.3s

```
Orders.columns = Orders.columns.str.strip()
Returns.columns = Returns.columns.str.strip()
People.columns = People.columns.str.strip()

Orders = Orders.merge>Returns, on='Order ID', how='left')
Orders['Returned'] = Orders['Returned'].fillna('No')
Orders = Orders.merge(People, on='Region', how='left')

Orders['Order Date'] = pd.to_datetime(Orders['Order Date'])
Orders['Ship Date'] = pd.to_datetime(Orders['Ship Date'])
```

✓ 0.0s

function สำหรับการ plot grap

```
sns.set_style("whitegrid")
def plot_sales_profit_from_summary(summary, group_col, title_suffix=None):

    x_labels = summary[group_col]
    sales = summary['Sales']
    profit = summary['Profit']

    plt.figure(figsize=(8,5))
    bar_width = 0.4
    x = range(len(x_labels))

    plt.bar(x, sales, width=bar_width, label='Sales', color='skyblue')
    plt.bar([i + bar_width for i in x], profit, width=bar_width, label='Profit', color='limegreen')

    # Annotation หัวเลขบน Bar
    for i in range(len(x_labels)):
        plt.text(i, sales[i]+max(sales)*0.01, f'{sales[i]:.0f}', ha='center', va='bottom', fontsize=9)
        plt.text(i + bar_width, profit[i]+max(profit)*0.01, f'{profit[i]:.0f}', ha='center', va='bottom', fontsize=9)

    plt.xticks([i + bar_width/2 for i in x], x_labels)
    plt.title(f'Sales and Profit by {group_col}' + (f' - {title_suffix}' if title_suffix else ''))
    plt.ylabel('Amount')
    plt.xlabel(group_col)
    plt.legend()
    plt.show()

    plt.figure(figsize=(6,6))
    plt.pie(sales, labels=x_labels, autopct='%1.1f%%', colors=sns.color_palette('pastel'), startangle=140)
    plt.title(f'Sales Distribution by {group_col}' + (f' - {title_suffix}' if title_suffix else ''))
    plt.show()

    plt.figure(figsize=(8,5))
    for i, row in summary.iterrows():
        plt.plot([0, row['Sales']], [0, row['Profit']], marker='o', label=row[group_col])
        plt.text(row['Sales']*1.01, row['Profit']*1.01, row[group_col], fontsize=10)

    plt.title(f'Profit vs Sales by {group_col}' + (f' - {title_suffix}' if title_suffix else ''))
    plt.xlabel('Sales')
    plt.ylabel('Profit')
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.legend()
```

```
def plot_sales_and_profit(data, title_suffix=None):
    top_sales = data.sort_values('Sales', ascending=False).head(10)
    top_profit = data.sort_values('Profit', ascending=False).head(10)
    plt.figure(figsize=(10,5))
    sns.barplot(
        x='Sales',
        y=f'{title_suffix}',
        data=top_sales,
        palette='Set2'
    )
    plt.title(f"Top 10 {title_suffix} by Sales")
    plt.xlabel("Sales")
    plt.ylabel(f"{title_suffix}")
    plt.show()
```

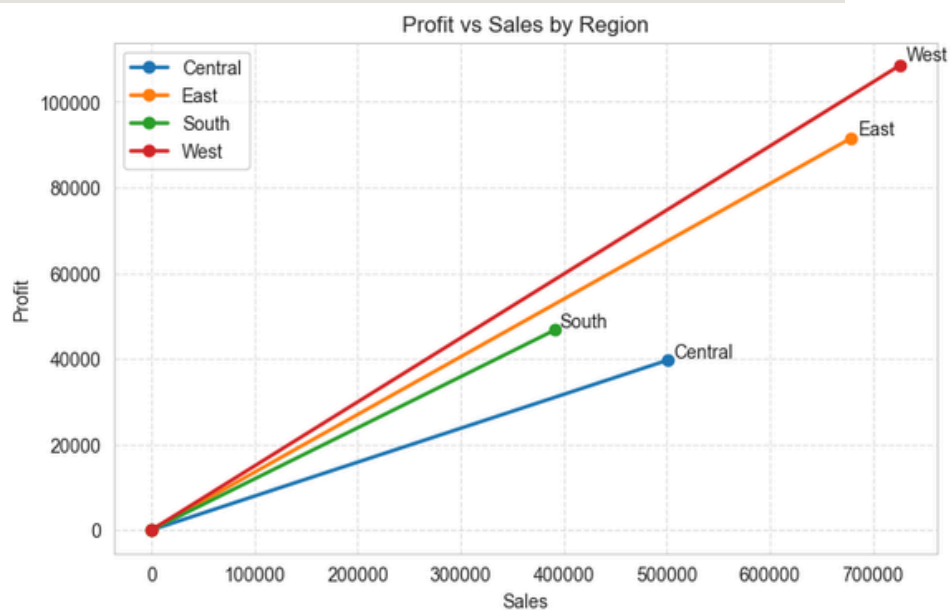
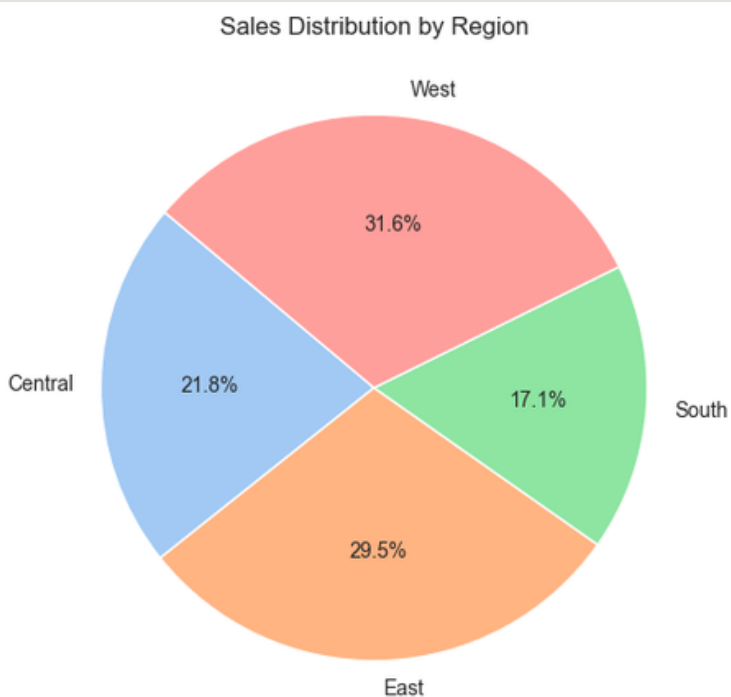
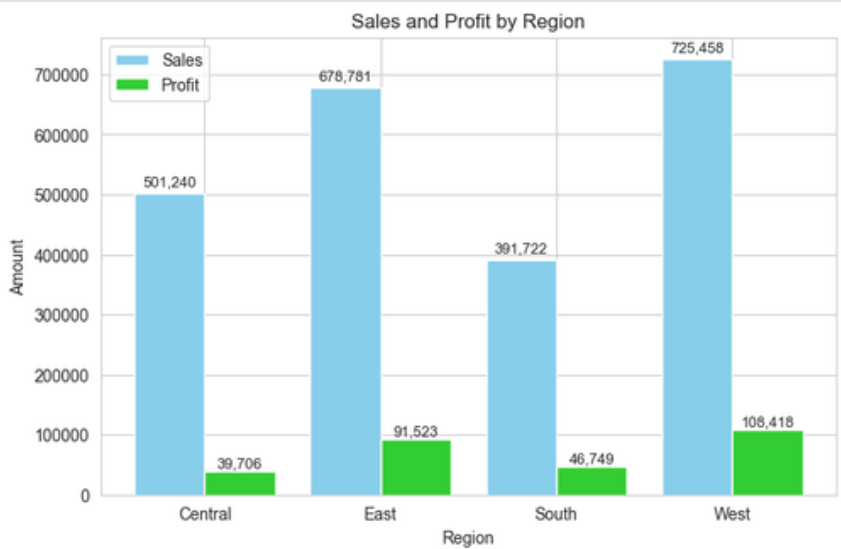
```
plt.figure(figsize=(10,5))
sns.barplot(
    x='Profit',
    y=f'{title_suffix}',
    data=top_profit,
    palette='Set2'
)
plt.title(f"Top 10 {title_suffix} by Profit")
plt.xlabel("Profit")
plt.ylabel(f"{title_suffix}")
plt.show()
```

Sales/Profit by Region

```
# --- Sales/Profit by Region --- #
sales_region = Orders.groupby('Region').agg({
    'Sales':'sum',
    'Profit':'sum',
    'Order ID':'nunique'
}).rename(columns={'Order ID':'Total Orders'}).reset_index()

plot_sales_profit_from_summary(sales_region, 'Region')

sales_region
```

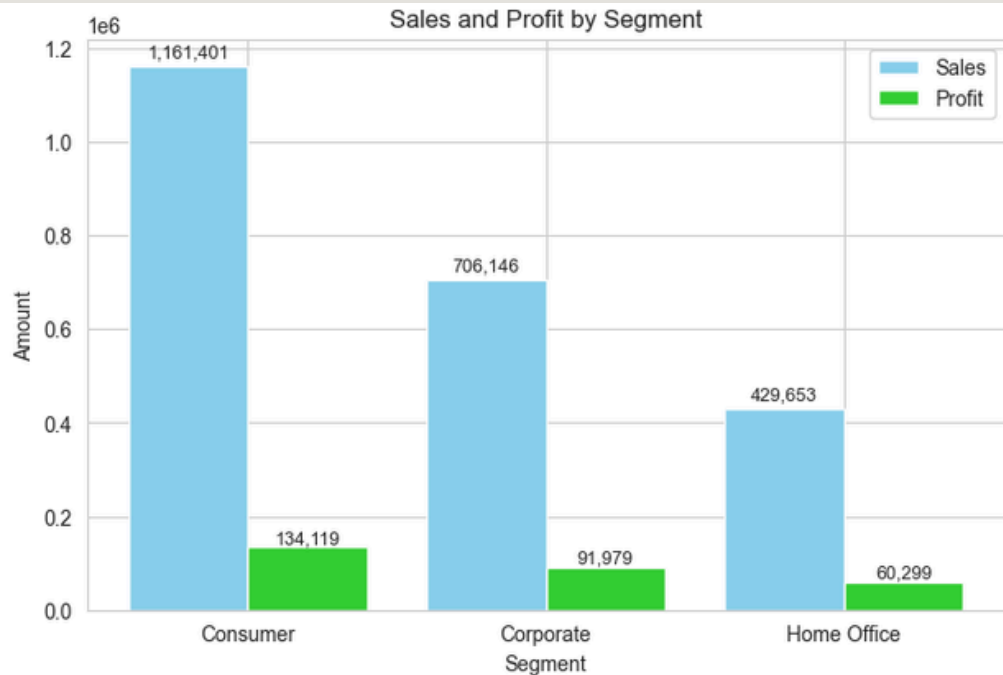


	Region	Sales	Profit	Total Orders
0	Central	501239.8908	39706.3625	1175
1	East	678781.2400	91522.7800	1401
2	South	391721.9050	46749.4303	822
3	West	725457.8245	108418.4489	1611

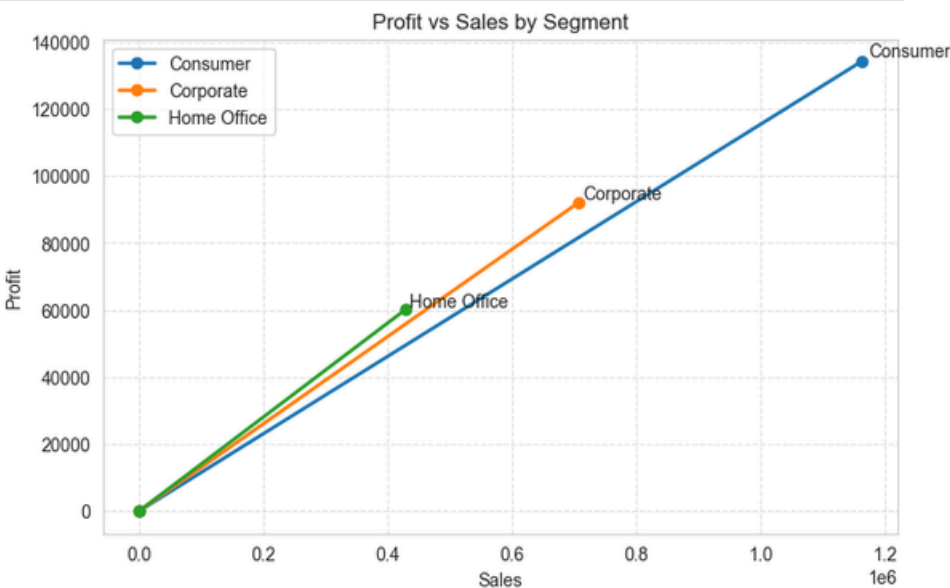
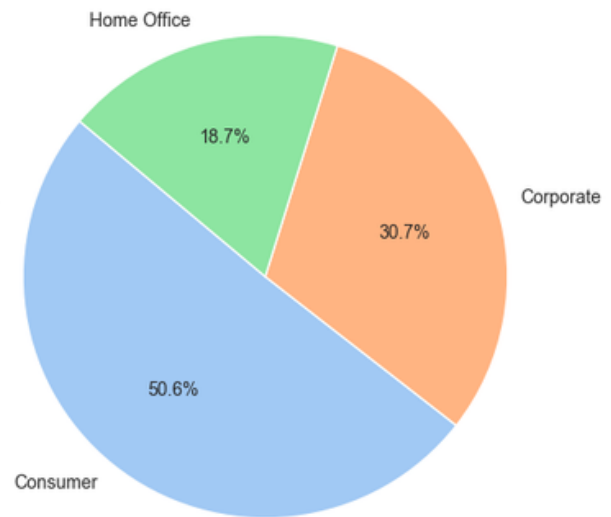
Sales/Profit by Segment

```
sales_segment = Orders.groupby('Segment').agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Order ID': 'nunique'
}).rename(columns={'Order ID': 'Total Orders'}).reset_index()

plot_sales_profit_from_summary(sales_segment, 'Segment')
sales_segment
```



Sales Distribution by Segment



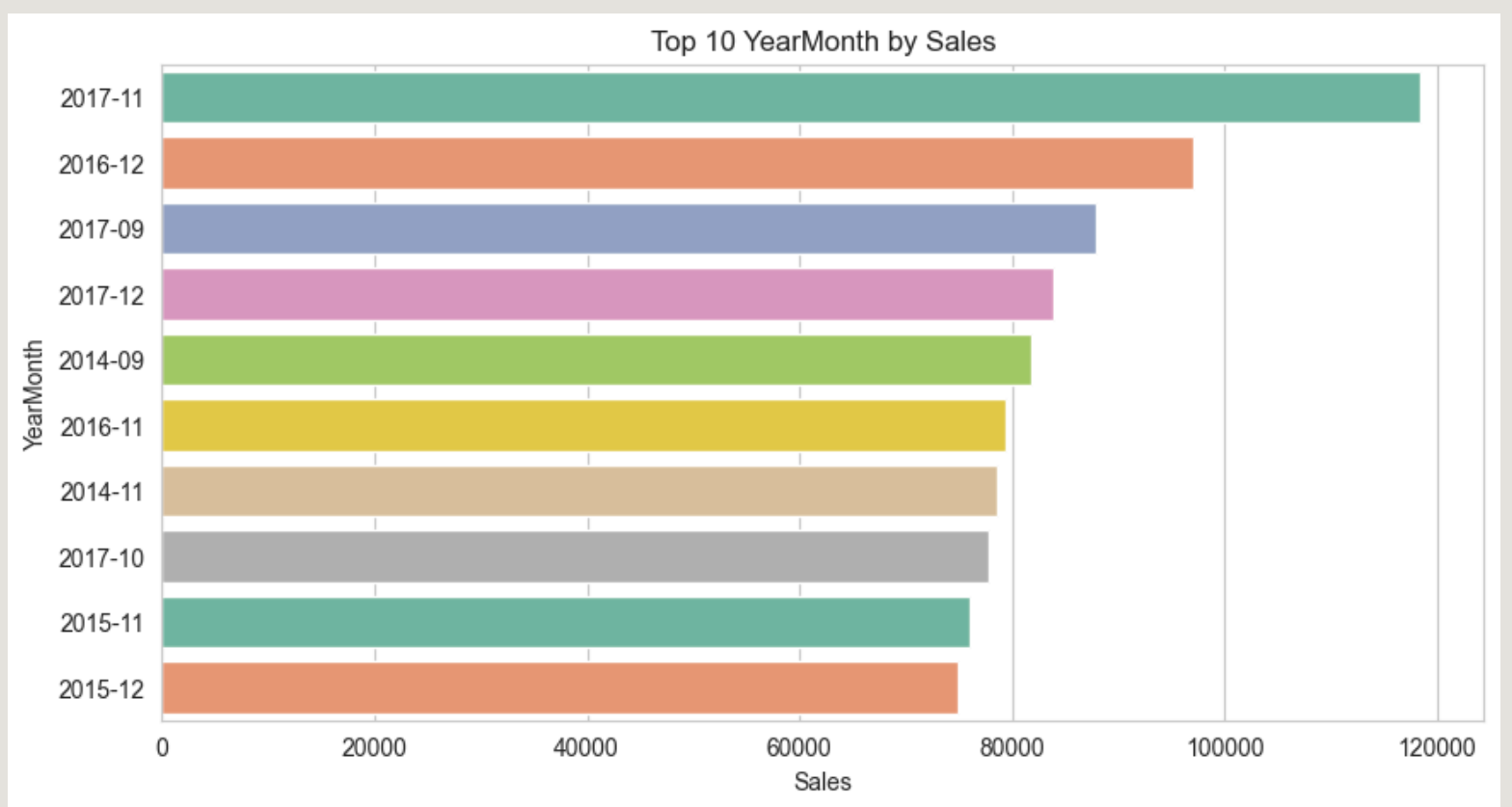
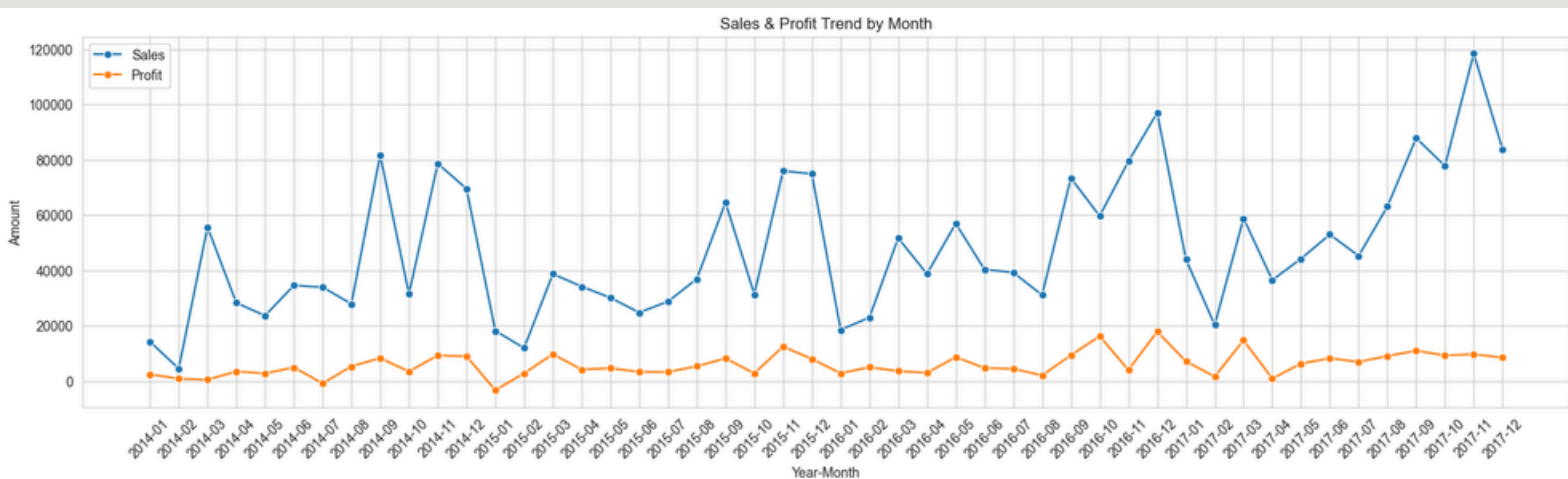
	Segment	Sales	Profit	Total Orders
0	Consumer	1.161401e+06	134119.2092	2586
1	Corporate	7.061464e+05	91979.1340	1514
2	Home Office	4.296531e+05	60298.6785	909

Trend by Month/Year

```
Orders['YearMonth'] = Orders['Order Date'].dt.to_period('M')
trend = Orders.groupby('YearMonth').agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Order ID': 'nunique'
}).rename(columns={'Order ID': 'Total Orders'}).reset_index()

trend['YearMonth_str'] = trend['YearMonth'].astype(str)
plt.figure(figsize=(20,5))
sns.lineplot(x='YearMonth_str', y='Sales', data=trend, marker='o', label='Sales')
sns.lineplot(x='YearMonth_str', y='Profit', data=trend, marker='o', label='Profit')
plt.xticks(rotation=45)
plt.title("Sales & Profit Trend by Month")
plt.xlabel("Year-Month")
plt.ylabel("Amount")
plt.legend()
plt.show()

plot_sales_and_profit(trend, 'YearMonth')
trend[['YearMonth', 'Sales', 'Profit', 'Total Orders']].head()
```

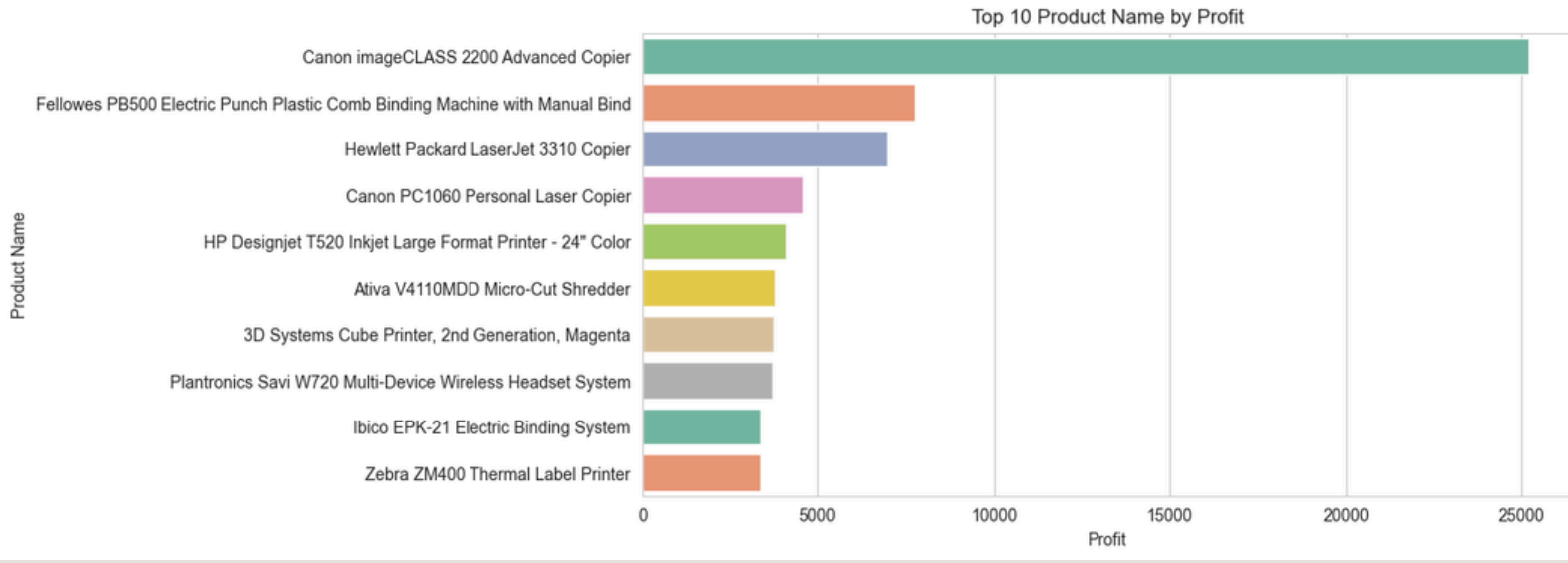
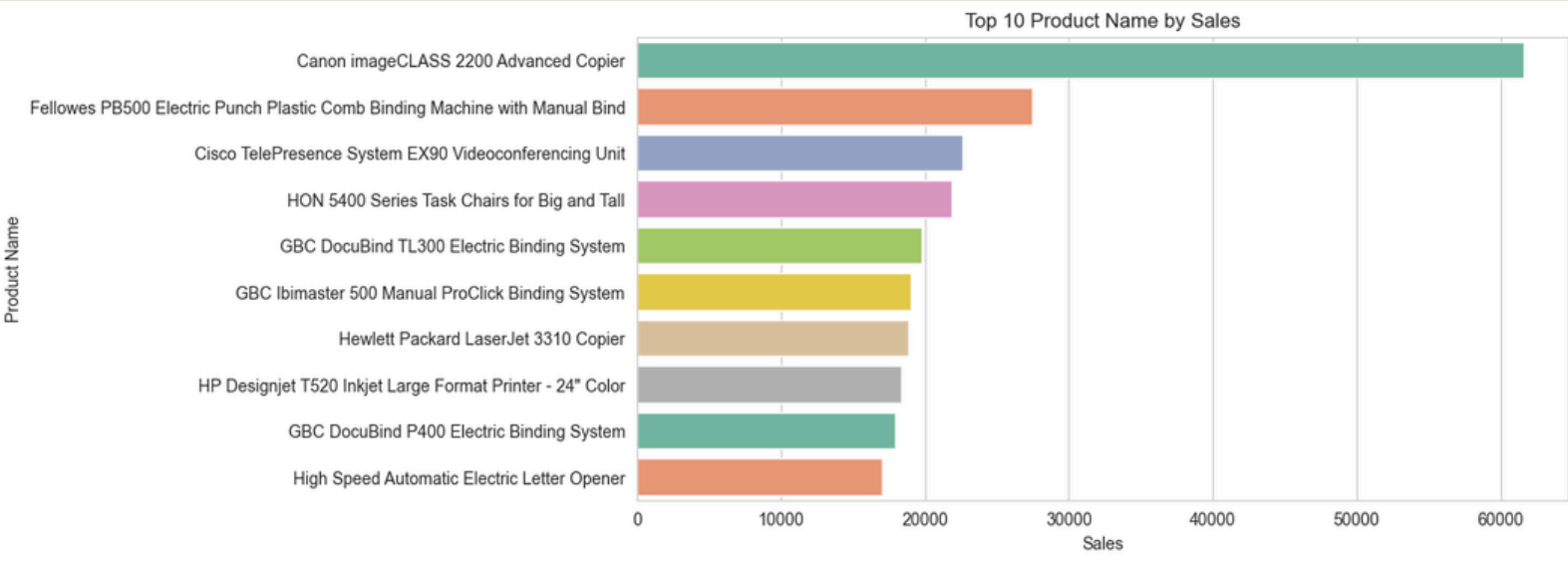


Sales/Profit by Products

```
products = Orders.groupby('Product Name').agg({
    'Sales': 'sum',
    'Profit': 'sum',
    'Order ID': 'nunique'
}).rename(columns={'Order ID': 'Total Orders'}).sort_values('Sales', ascending=False).reset_index()

plot_sales_and_profit(products, 'Product Name')

products
```



	Product Name	Sales	Profit	Total Orders
0	Canon imageCLASS 2200 Advanced Copier	61599.824	2.519993e+04	5
1	Fellowes PB500 Electric Punch Plastic Comb Bin...	27453.384	7.753039e+03	10
2	Cisco TelePresence System EX90 Videoconferenci...	22638.480	-1.811078e+03	1
3	HON 5400 Series Task Chairs for Big and Tall	21870.576	3.979039e-13	8
4	GBC DocuBind TL300 Electric Binding System	19823.479	2.233505e+03	11
...	...	...	...	...
1845	Avery Hi-Liter Pen Style Six-Color Fluorescent...	7.700	3.157000e+00	1
1846	Grip Seal Envelopes	7.072	2.386800e+00	1
1847	Xerox 20	6.480	3.110400e+00	1
1848	Avery 5	5.760	2.822400e+00	1
1849	Eureka Disposable Bags for Sanitaire Vibra Gro...	1.624	-4.466000e+00	1

Web Scraping Python

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

# ดึงข้อมูลจากเว็บไซต์
response = requests.get("https://www.myhora.com/%E0%B8%AB%E0%B8%A7%E0%B8%A2/%E0%B8%AA%E0%B8%
soup = BeautifulSoup(response.text, 'html.parser')

# ดึงข้อมูลที่ต้องการจากเว็บไซต์
data = []
temp_row = []
a = soup.find_all('div', class_='colx')
del a[:28]

for i, div in enumerate(a, start=1):
    temp_row.append(div.text.strip())
    if i % 12 == 0:
        data.append(temp_row)
        temp_row = []

if temp_row:
    data.append(temp_row)

df = pd.DataFrame(data)

df.to_excel("หอย.xlsx", index=False, header=False)
```

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	1	เมษายน	เม.ย.	2567	67	803481	81	481	90	122 809 5 90		122 809 559 947	
2	16	มีนาคม	มี.ค.	2567	67	997626	26	626	78	509 571 3 78		509 571 329 794	
3	1	มีนาคม	มี.ค.	2567	67	253603	03	603	79	900 975 3 79		900 975 382 703	
4	16	กุมภาพันธ์	ก.พ.	2567	67	941395	95	395	43	056 330 3 43		056 330 375 587	
5	1	กุมภาพันธ์	ก.พ.	2567	67	607063	63	063	09	454 943 5 09		454 943 544 591	
6	17	มกราคม	ม.ค.	2567	67	105979	79	979	61	429 931 1 61		429 931 196 635	
7	30	ธันวาคม	ธ.ค.	2566	66	625544	44	544	89	600 648 4 89		600 648 456 882	
8	16	ธันวาคม	ธ.ค.	2566	66	356757	57	757	85	058 410 5 85		058 410 584 964	
9	1	ธันวาคม	ธ.ค.	2566	66	251097	97	097	91	055 265 0 91		055 265 092 280	
10	16	พฤศจิกายน	พ.ย.	2566	66	557990	90	990	14	346 412 7 14		346 412 778 961	
11	1	พฤศจิกายน	พ.ย.	2566	66	743951	51	951	63	335 913 0 63		335 913 019 349	
12	16	ตุลาคม	ต.ค.	2566	66	931446	46	446	44	167 398 2 44		167 398 272 970	
13	1	ตุลาคม	ต.ค.	2566	66	727202	02	202	66	324 355 4 66		324 355 426 615	
14	16	กันยายน	ก.ย.	2566	66	320812	12	812	46	037 699 0 46		037 699 057 344	
15	1	กันยายน	ก.ย.	2566	66	915478	78	478	91	521 596 2 91		521 596 291 692	
16	16	สิงหาคม	ส.ค.	2566	66	471782	82	782	67	431 739 7 67		431 739 737 742	
17	31	กรกฎาคม	ก.ค.	2566	66	260453	53	453	11	268 708 3 11		268 708 387 601	
18	16	กรกฎาคม	ก.ค.	2566	66	169530	30	530	62	261 384 0 62		261 384 066 780	
19	1	กรกฎาคม	ก.ค.	2566	66	922605	05	605	16	281 867 4 16		281 867 491 947	
20	16	มิถุนายน	มิ.ย.	2566	66	264872	72	872	30	519 628 2 30		519 628 202 874	
21	1	มิถุนายน	มิ.ย.	2566	66	125272	72	272	09	000 681 3 09		000 681 386 971	
22	16	พฤษภาคม	พ.ค.	2566	66	132903	03	903	99	678 739 6 99		678 739 693 731	
23	2	พฤษภาคม	พ.ค.	2566	66	843019	19	019	65	500 780 1 65		500 780 187 269	
24	16	เมษายน	เม.ย.	2566	66	984906	06	906	71	670 678 5 71		670 678 551 797	
25	1	เมษายน	เม.ย.	2566	66	087907	07	907	99	111 914 2 99		111 914 290 698	
26	16	มีนาคม	มี.ค.	2566	66	025873	73	873	73	420 800 3 73		420 800 355 544	
27	1	มีนาคม	มี.ค.	2566	66	417652	52	652	55	577 919 7 55		577 919 748 984	
28	16	กุมภาพันธ์	ก.พ.	2566	66	590417	17	417	80	195 664 3 80		195 664 377 523	
29	1	กุมภาพันธ์	ก.พ.	2566	66	297411	11	411	92	181 789 1 92		181 789 101 664	
30	17	มกราคม	ม.ค.	2566	66	812519	19	519	47	389 443 5 47		389 443 564 849	
31	30	ธันวาคม	ธ.ค.	2565	65	157196	96	196	58	007 522 2 58		007 522 250 425	
32	16	ธันวาคม	ธ.ค.	2565	65	845093	93	093	14	411 912 5 14		411 912 593 855	
33	1	ธันวาคม	ธ.ค.	2565	65	375805	05	805	08	170 786 4 08		170 786 409 421	
34	16	พฤศจิกายน	พ.ย.	2565	65	121789	89	789	64	532 722 1 64		532 722 157 973	
35	1	พฤศจิกายน	พ.ย.	2565	65	913106	06	106	70	722 839 3 70		722 839 343 922	
36	16	ตุลาคม	ต.ค.	2565	65	613106	06	106	15	037 158 6 15		037 158 606 799	

Certificate

OF COMPLETION

ກູດລ ວຽນຸຕຣ

has successfully completed

Get started with Programming ຮວມສິ່ງທີ່ຕ້ອງຮູ້ ກ່ອນເດີນສູ່ເສັ້ນທາງໂປຣແກຣມເມື່ອ

an online course offered by FutureSkill

03/04/2024



Futureskill has confirmed the identity of this individual and their participation in the course

Futureskill.co

Certificate

OF COMPLETION

ກູດລ ວຽນຸຕຣ

has successfully completed

ຈັດການຂໍ້ມູນລູກຢ່າງມີປະສິດທິພາບ ພ້ອມກັບວິເຄາະຂໍ້ມູນທີ່ມີໃນໄມ້ທີ່ອັດໄດ້ດ້ວຍ SQL Command

an online course offered by FutureSkill

19/12/2023



Futureskill has confirmed the identity of this individual and their participation in the course

Futureskill.co

Certificate

OF COMPLETION

ກູດລ ວຽນຸຕຣ

has successfully completed

UpSkill Python Programming ເລີ່ມທາງສູ່ Developer

an online course offered by FutureSkill

11/01/2024



Futureskill has confirmed the identity of this individual and their participation in the course

Futureskill.co

ภูวดล วรรณตร

ได้ผ่านการเรียนออนไลน์ตามเกณฑ์การวัดผลในรายวิชา

ความรู้พื้นฐานเกี่ยวกับบักดาตา (10 ชั่วโมงการเรียนรู้)

พัฒนารายวิชาโดย มหาวิทยาลัยสุโขทัยธรรมาธิราช

ผศ.ดร. วุฒิชัย ร่มสายหยุด
สาขาวิชาวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยสุโขทัยธรรมาธิราช

ภูวดล วรรณตร

ได้ผ่านการเรียนออนไลน์ตามเกณฑ์การวัดผลในรายวิชา

วิทยาการข้อมูลและการประยุกต์ใช้ (30 ชั่วโมงการเรียนรู้)

พัฒนารายวิชาโดย จุฬาลงกรณ์มหาวิทยาลัย

ผศ.ดร.ณัฐวุฒิ หนูโพธิ์วงษ์
รองศาสตราจารย์
ศาสตร์ จุฬาลงกรณ์มหาวิทยาลัย

ภูวดล วรรณตร

ได้ผ่านการเรียนออนไลน์ตามเกณฑ์การวัดผลในรายวิชา

การวิเคราะห์เชิงสถิติกับมหาข้อมูล (10 ชั่วโมงการเรียนรู้)

พัฒนารายวิชาโดย มหาวิทยาลัยรังสิต

ผศ. ดร. เชษฐพงศ์ ศรีสุอาน
คณบดี, ว. นวัตกรรมดิจิทัลและเทคโนโลยีสารสนเทศ
มหาวิทยาลัยรังสิต

ภูวดล วรรณตร

ได้ผ่านการเรียนออนไลน์ตามเกณฑ์การวัดผลในรายวิชา

ภาษาอังกฤษสำหรับเทคโนโลยีสารสนเทศ (10 ชั่วโมงการเรียนรู้)

พัฒนารายวิชาโดย มหาวิทยาลัยขอนแก่น

ผศ.ดร.กรวิภา พูลผล
รองคณบดีฝ่ายแผนและสารสนเทศ
คณะมนุษยศาสตร์และสังคมศาสตร์ มหาวิทยาลัยขอนแก่น



หลักสูตรวิทยาการการจัดการข้อมูล
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ
ขอมอบเกียรติบัตรฉบับนี้ให้ไว้เพื่อแสดงว่า

นาย ภูวดล วรบุตร

ผ่านการอบรม หัวข้อ

Google Data Studio กับการวิเคราะห์ข้อมูลเชิงธุรกิจ

โครงการพัฒนาสมรรถนะวิชาชีพนักศึกษาประจำหลักสูตร

วันที่ ๑๐ กุมภาพันธ์ พ.ศ. ๒๕๖๖

(ดร.จุริรา คงนุ้ย)

ประธานหลักสูตรวิทยาการการจัดการข้อมูล

Made for free with Certify'em



สำนักวิทยบริการและเทคโนโลยีสารสนเทศ
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ

ขอมอบวุฒิบัตรให้ไว้เพื่อแสดงว่า

นาย ภูวดล วรบุตร

เข้าร่วมอบรมแนะนำการใช้งาน

Google Workspace for Education Plus

ให้ไว้ ณ วันที่ 30 มกราคม 2566

ดร.เอกชัย เนาวนิช

ผู้อำนวยการสำนักวิทยบริการและเทคโนโลยีสารสนเทศ



สำนักวิทยบริการและเทคโนโลยีสารสนเทศ
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ

ขอมอบวุฒิบัตรให้ไว้เพื่อแสดงว่า

นาย ภูวดล วรบุตร

ได้เข้าร่วมเสริมสร้างความฉลาดรู้ด้านดิจิทัลในอาเซียน

: ASEAN DIGITAL LITERACY PROGRAMME (ADLP) จำนวน 3 ชม.

ให้ไว้ ณ วันที่ 25 มกราคม พ.ศ. 2566

นายเอกชัย เนาวนิช

ผู้อำนวยการสำนักวิทยบริการและเทคโนโลยีสารสนเทศ



thank you